



UNIVERSITÀ DI PISA

Master Degree in Computer Science
Curriculum: Artificial Intelligence

Computational Mathematics for Learning and Data Analysis

Academic Year 2025/2026

Group Number: 48
Project Number: 4 ML

Giorgio Chelli

Massimo Monai

Project Track:

(*M1*) is a Neural Network with topology and activation function of choice (differentiable or not), but mandatory *L1* regularization;

(*M2*) is a standard linear regression (Least Squares);

(*A1*) is a standard momentum descent (Heavy Ball) approach applied to (*M1*);

(*A2*) is an algorithm of the class of Deflected Subgradient methods applied to (*M1*);

(*A3*) is a basic version of the direct linear least squares solver of choice (Normal Equations, QR, or SVD) applied to (*M2*).

Contents

1	Project Introduction	3
1.1	Learning problem	3
1.2	Algorithm A1	4
1.2.1	The momentum term definition	4
1.2.2	L1 Regularization with Gradient-based methods	5
1.2.3	L1 Regularization with Leaky ReLU	5
1.2.4	Adaptive Stepsize Rule Integration for Fair Comparison	6
1.3	Algorithm A2	6
1.3.1	Stepsize Rule	7
1.3.2	Deflection Rule	8
1.3.3	Theoretical convergence	10
1.4	Algorithm A3	10
1.4.1	Factorization and Resolution	10
1.4.2	Householder QR factorization implementation	11
1.4.3	Computational cost	12
2	Expected results	14
2.1	Empirical estimation of the optimal value	14
2.2	Algorithm A1	15
2.3	Algorithm A2	17
2.4	Algorithm A3	18
3	Experimental set-up	21
3.1	Dataset	21
3.1.1	Pre-processing	21
3.1.2	Collinearity and conditioning	22
3.2	Model Selection	23
3.2.1	Model selection for M1	23
3.2.2	Model selection for M2	23
3.3	Algorithms Hyperparameters Selection	23
3.3.1	Stepsize rules hyperparameters	24
3.3.2	Algorithm A1: Heavy Ball	29

3.3.3	Algorithm A2: Volume algorithm	33
4	Results and comments	34
4.1	Model M1: Heavy Ball vs Volume Algorithm	34
4.1.1	Estimation of the best-known value $\bar{f}_\lambda$	35
4.1.2	Stepsize Rules: ColorTV vs SGPTL	37
4.1.3	Verification of the Expected Theoretical Properties of Subgradient Methods	40
4.2	Model M2: Considerations on QR direct method	45
4.2.1	Computational performance	45
4.2.2	Numerical stability and Conditioning verification	46
4.2.3	Effect of L2 Regularization	47

Chapter 1

Project Introduction

1.1 Learning problem

The problem of the project is a supervised multivariate regression task: given the dataset **ML-CUP25-TR** with samples $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}^c$, we want to learn a function $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^c$ that minimizes the objective function of the form:

$$\min_{\theta} L(\theta) = \frac{1}{n} \sum_{i=1}^n \|f_\theta(x_i) - y_i\|_2^2 + \lambda \cdot \Omega(\theta) \quad (1.1)$$

where λ is the regularization coefficient and $\Omega(\theta)$ is the regularization term.

Model M1 The *M1* model requires the implementation of *L1* regularization, so the objective function will be of the form:

$$L_{M1}(\theta) = \frac{1}{n} \sum_{i=1}^n \|f_\theta(x_i) - y_i\|_2^2 + \lambda \cdot \|\theta\|_1 \quad (1.2)$$

The *L1* norm is defined as:

$$\sum_{i=1}^{t_\theta} |\theta_i| \quad (1.3)$$

where t_θ is the dimension of the parameter vector θ , while the sum of their absolute values introduces non-differentiability into the learning problem.

Because the non-differentiability property introduced by *L1* we use algorithms based on sub-gradient, with the comparison between the **Deflected Subgradient** (A2) and the **Heavy Ball** approach (A1), normally used for differentiable losses, which plays a central role in this report.

The *M1* model is a Feedforward Neural Network (Multi-Layer Perceptron) structured as follows:

- **Topology:** Two hidden layers with $h1$ and $h2$ neurons and an output layer with $c = 4$ linear neurons, one for each target of the dataset.

- **Activation function:** *LeakyReLU* have been used as activation function for hidden layers.
- **Initialization:** He initialization for the *LeakyReLU* function [8].

Model M2 The *M2* model is a multivariate linear regression model with standard *L2* regularization (Ridge Regression). The objective function to be minimized takes the following matrix form:

$$L_{M2}(\Theta) = \frac{1}{n} \|\mathbf{X}\Theta - \mathbf{Y}\|_F^2 + \lambda \|\Theta\|_F^2 \quad (1.4)$$

where $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ represents the input data matrix (extended with a unique column to incorporate the bias term), $\mathbf{Y} \in \mathbb{R}^{n \times c}$ is the dataset target matrix, $\Theta \in \mathbb{R}^{(d+1) \times c}$ is the matrix of parameters to be learned and $\|\cdot\|_F$ denotes the Frobenius norm.

Unlike the *M1* model, the objective function L_{M2} is strictly convex and infinitely differentiable everywhere. The presence of the quadratic regularization term $\lambda \|\Theta\|_F^2$ offers two advantages:

- **Capacity Control:** Reduces the variance of the parameters by reducing their magnitude, preventing overfitting phenomena.
- **Numerical Stability:** Makes the coefficient matrix associated with the linear system positive definite even in the presence of strong collinearity between features, ensuring the existence and uniqueness of the solution.

1.2 Algorithm A1

1.2.1 The momentum term definition

The standard Gradient Descent update rule for each parameter θ at iteration $k + 1$ is:

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k) \quad (1.5)$$

In addition to this, Heavy Ball introduces a new term representing the physical concept of velocity and momentum:

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k) + \boxed{\alpha v_k}, \quad 0 < \alpha < 1, \quad (1.6)$$

where:

- α is a coefficient that controls the strength of the momentum;
- $v_k = \theta_k - \theta_{k-1}$ is the velocity term, calculated considering the current optimization step θ_k and the previous optimization steps θ_{k-1} .

In other terms, given the velocity at $k + 1$ iteration as:

$$\mathbf{v}_{k+1} = \alpha \mathbf{v}_k - \eta \nabla f(\theta_k) \quad (1.7)$$

by recursively expanding the velocity terms at past iterations, it can be rewritten as:

$$\mathbf{v}_{k+1} = -\eta \nabla f(\theta_k) + \alpha (-\eta \nabla f(\theta_{k-1}) + \alpha \mathbf{v}_{k-1}), \quad (1.8)$$

$$= -\eta \nabla f(\theta_k) - \eta \alpha \nabla f(\theta_{k-1}) + \alpha^2 \mathbf{v}_{k-1}, \quad (1.9)$$

$\vdots$

$$= -\eta \sum_{i=0}^k \alpha^i \nabla f(\theta_{k-i}) \quad (1.10)$$

assuming the initial velocity is zero, i.e. $\mathbf{v}_0 = 0$.

Hence, the velocity is implemented as an exponentially weighted average of all previous gradient computations, where the contribution of older gradients decreases geometrically according to the factor α^i .

1.2.2 L1 Regularization with Gradient-based methods

$L1$ regularization is not designed to be optimized through plain gradient-based methods, since these methods require the loss function to be differentiable everywhere, while the absolute value function $|\theta|$, which constitutes the $L1$ penalty term, is non-differentiable at $\theta = 0$.

In practice, this obstruction is bypassed by replacing the gradient of the penalty term with a *subgradient*:

$$\partial|\theta| = \begin{cases} 1 & \theta > 0, \\ -1 & \theta < 0, \\ [-1, 1] & \theta = 0, \end{cases}$$

which is added to the backpropagated gradient before applying the momentum update.

1.2.3 L1 Regularization with Leaky ReLU

Another source of non-smoothness comes from the interaction between $L1$ regularization and the Leaky ReLU activation, defined as:

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0, \\ \zeta x & x < 0, \end{cases} \quad 0 < \zeta < 1, \quad (1.11)$$

which is non-differentiable only at the single point $x = 0$. As $L1$ pushes a growing subset of weights toward small magnitudes, the corresponding pre-activations are increasingly likely

to lie close to this non-differentiable kink. So, the optimization landscape of $A1$ combines two distinct sources of non-smoothness: the $L1$ penalty at $\theta = 0$, and the Leaky ReLU kink when pre-activation $net = 0$. However, both have been handled in practice through a chosen subgradient convention. In the end, Leaky ReLU guarantees a non-zero gradient on the negative side, so this interaction does not risk fully silencing a unit’s contribution to the backward pass. The value of ζ is set to 0.01.

1.2.4 Adaptive Stepsize Rule Integration for Fair Comparison

Our implementation of Heavy Ball integrates an adaptive stepsize rule natively associated with subgradient approaches, such as the ColorTV or SGPTL rules (introduced in Sec. 1.3.1). The main motivation for this architectural choice is to ensure the cleanest, most equitable comparison between Algorithm $A1$ (Heavy Ball) and Algorithm $A2$ (Deflected Subgradient). By unifying the stepsize rule across both algorithms, we can isolate the effect of the update direction.

The mechanism to update the adaptive learning rate operates as follows:

- **Full-Batch Gradient Evaluation:** A full pass on the normalized training set is performed to compute the current objective loss (L_{curr}) and the full-batch gradient (g_{full}) of the non-smooth landscape.
- **ColorTV Heuristic Update:** The algorithm evaluates the success of the previous step by comparing the loss reduction (Δf) and the inner product of the previous parameter update against g_{full} . The base scaling factor η is then updated according the ColorTV stepsize rule.
- **Epoch Stepsize Calculation:** The scaled η is used to compute the epoch’s effective stepsize (η_{epoch}) via the adapted Polyak rule:

$$\eta_{\text{epoch}} = \max \left(\eta \frac{L_{\text{curr}} - f_{\text{lev}}}{\|g_{\text{full}}\|^2 + \epsilon_{\text{tol}}}, 0 \right) \quad (1.12)$$

where f_{lev} is the dynamically updated target level of the loss and $\epsilon_{\text{tol}} = 10^{-9}$ is a small stabilization constant to prevent zero-division.

- **Mini-Batch Application:** The computed η_{epoch} is subsequently passed into the mini-batch gradient update loop, where it scales the gradient contribution before the standard Heavy Ball momentum velocity (v_k) is applied to the weights.

1.3 Algorithm A2

The $A2$ algorithm implements the deflected subgradient method for optimizing the non-smooth $M1$ model. The update direction is calculated as a convex combination between the current subgradient g_i and the previous direction d_{i-1} , weighted by the deflection parameter $\gamma_i \in [0, 1]$:

$$d_i = \gamma_i \cdot g_i + (1 - \gamma_i) \cdot d_{i-1} \quad (1.13)$$

The subgradient algorithm can follow several implementations, the common feature of which is the need to define two update rules for the main parameters:

- η : the stepsize for updating weights, defined through the **Stepsize Rule**;
- γ : the deflection parameter for calculating the deflected subgradient, defined through the **Deflection Rule**;

1.3.1 Stepsize Rule

In our project, two **Stepsize Rule** were tested:

- **ColorTV**: The ColorTV rule, introduced in the [2] paper, but developed in this work following the indications of the paper [1], which uses Polyak's rule [3, Eq.(2), p.15] as the stepsize update rule:

$$\eta = \beta \cdot \frac{f(x) - f_{lev}}{\|g\|^2} \quad (1.14)$$

- **SGPTL (Subgradient with Polyak-Type Level)** [5, Sect.3.1, Eq.(3.2), p.364]: This rule for calculating the stepsize η , compared to the classic Polyak rule, calculates the stepsize as:

$$\eta = \beta \cdot \frac{f(x) - (f_{ref} - \delta)}{\|g\|^2} \quad (1.15)$$

Both formulas have been adapted for the deflected subgradient, replacing the subgradient $\|g\|^2$ in the denominator with the deflected direction, calculated as:

$$d = \gamma \cdot g + (1 - \gamma) \cdot d_{prev} \quad (1.16)$$

The denominator is then replaced with $\|d\|^2$.

ColorTV The **ColorTV Rule** is based on reducing the value of the objective function $\Delta f = \bar{f}_{i-1} - f_i$ by f and on the dot product $d_i \cdot g_i$ to estimate the success of the last step.

The ColorTV Rule is based on reducing the value of the objective function

$$\Delta f = \bar{f}_{i-1} - f_i$$

by f and on the dot product $d_i \cdot g_i$ to estimate the success of the last step.

The iteration i is defined as:

- **Green** if $d_i \cdot g_i > \rho_{CT}$ and $\Delta f > \rho_{CT} \cdot \max\{|f_i^{\text{rec}}|, 1\}$
- **Yellow** if $d_i \cdot g_i < \rho_{CT}$ and $\Delta f \geq \rho_{CT}$
- **Red** otherwise

where ρ_{CT} is the tolerance.

A step `green` is a good step while a step `red` suggests a parameter η that is too large. Three parameters c_g, c_y and c_r and three counters n_g, n_y and n_r of the number of consecutive iterations of type `green`, `yellow`, `red` are used respectively. These parameters and counters are used to update the β_i parameter, which allows you to accelerate or decelerate the stepsize:

- If $n_g \geq c_g$, then $\beta_i = \min\{2, 2 \cdot \beta_{i-1}\}$;
- If $n_y \geq c_y$, then $\beta_i = \min\{2, 1.1 \cdot \beta_{i-1}\}$;
- If $n_r \geq c_r$, then $\beta_i = \max\{5 \cdot 10^{-4}, 0.67 \cdot \beta_{i-1}\}$;
- If none of the above cases occur, then $\beta_i = \beta_{i-1}$.

A fundamental role in the procedure is therefore played by the initialization β_0 which allows us to define the initial descent rate.

To estimate f_{lev} , the loss at the previous iteration was used, updating the value if $f_{i-1} < 1.05 \cdot f_{lev}$ with:

$$f_{lev} = f_{i-1} - 0.05 \cdot |f_{lev}| \quad (1.17)$$

SGPTL `SGPTL Rule` [5, Sect.3.1, Eq.(3.2), p.364] focuses on finding the optimal value of the parameter f_{best} , estimated as $f_{ref} - \delta$, where δ is a hyperparameter iteratively updated based on the loss descent rate.

The SGPTL rule was born for the subgradient without deflection, but we tested its application also for the deflected subgradient using the deflected direction $\|d\|^2$ as the denominator.

During the procedure the following variables are used:

- f_{ref} : Reference value used in Polyak's rule [3, Eq.(2), p.15] in combination with δ to predict the target level (ideally) $f_{ref} - \delta \approx f_{best}$.
- $\bar{f}$: Stores the minimum value of f found at the current iteration i , and is used to update f_{ref} .
- r : Accumulation variable used to store the route taken in the loss space without achieving significant improvements.

1.3.2 Deflection Rule

As `Deflection Rule` we implemented the `Volume Algorithm`, introduced in [2, Theorem 1, p. 388] for convex optimization problems with subgradients. In the context of neural network training, theoretical convergence guarantees do not transfer directly, but the structure of the deflection rule represents a practical advantage over fixed-momentum methods, as confirmed by experimental results. The convex combination update is described in [2, Sec. 4, p. 391].

The parameter γ is in fact calculated as the optimal solution to the following univariate quadratic problem, inspired by the form *poor man's* of the proximal Bundle method master problem [1, Eq.26, p.598]:

$$\min_{\gamma \in [0,1]} \frac{\eta_{i-1}}{2} \|\gamma g_i + (1 - \gamma) d_{i-1}\|^2 + \gamma \sigma_i(\bar{\theta}_i) + (1 - \gamma) \varepsilon_{i-1}(\bar{\theta}_i), \quad (1.18)$$

where:

- η_{i-1} is the stepsize at the previous iteration, used as a stability weight.
- $\sigma_i(\bar{\theta} * i)$ and $\varepsilon * i - 1(\bar{\theta}_i)$ are the linearization errors at the current stability center $\bar{\theta} * i$, measuring how poorly the linear models constructed from g_i and $d * i - 1$, respectively, approximate the objective function locally.

Relaxing the constraint $\gamma \in [0, 1]$ the problem admits a closed-form solution:

$$\gamma_i^* = \frac{\varepsilon_{i-1}(\bar{\theta}_i) - \sigma_i(\bar{\theta}_i) - \eta_{i-1} d_{i-1}^\top (g_i - d_{i-1})}{\eta_{i-1} \|g_i - d_{i-1}\|^2}, \quad (1.19)$$

whose optimal solution, respecting the original constraint, is obtained by projecting γ_i^* on the interval $[0, 1]$.

As for the projection operation, the same variant proposed in the original paper of [1, Appendix, Section “Parameters of the DR”], defined by:

$$\gamma_i = \begin{cases} \gamma_{i-1}/10 & \text{if } \gamma_i^* \leq 10^{-8}, \\ \min\{\tau_i, 1\} & \text{if } \gamma_i^* \geq 1, \\ \gamma_i^* & \text{otherwise,} \end{cases} \quad (1.20)$$

where τ_i is an adaptive upper bound initialized with hyperparameter τ_0 and decremented every τ_p iterations by a factor $\tau_f \in (0, 1)$, subject to the lower bound $\tau_{\min}$.

This formulation, applied to our settings, had a critical bound in the first iterations: it generated negative values of γ_i^* from which the algorithm was unable to deviate, compromising subsequent convergence.

To overcome this problem, the following modified variant has been introduced:

$$\gamma_i = \begin{cases} 1 & \text{if } \gamma_i^* \leq 0, \\ \gamma_{i-1} & \text{if } \gamma_i^* \leq 10^{-8} \& \gamma_i^* > 0 \\ \min\{\tau_i, 1\} & \text{if } \gamma_i^* \geq 1, \\ \gamma_i^* & \text{otherwise,} \end{cases} \quad (1.21)$$

We impose $\gamma_i = 1$ when $\gamma_i^* \leq 0$. This happens frequently during the initial iterations, because in this phase deflection was not yet stably computable. So this condition ensure the correct start of the algorithm and then allowing the smooth transition to the standard update dynamics.

1.3.3 Theoretical convergence

In subgradient analysis, theoretical convergence in the case of the Lagrangian problem can be achieved through the **Stepsize-restricted** rule, which allows the subgradient step to be limited at each iteration [5, Section 3.2, Eq. 3.19, p. 372]. The constraint to be respected is the following:

$$\beta_i \leq \gamma_i \leq 1 \quad (1.22)$$

The **Stepsize-restricted** rule was also applied in the non-convex and non-smooth scenario, as a tool to prevent the algorithm from continuously jumping beyond the valleys of local optima, stabilizing the loss behavior on the training set.

1.4 Algorithm A3

Algorithm A3 solves the ridge regression problem associated with model $M2$ (Eq. 1.23) directly, via QR factorization, avoiding the explicit formation of the system of normal equations.

The problem to be solved is not a linear system of the form $A\theta = b$, but rather a regularized least squares problem:

$$\min_{\Theta} \frac{1}{n} \|X\Theta - Y\|_F^2 + \lambda \|\Theta\|_F^2 \quad (1.23)$$

with $X \in \mathbb{R}^{n \times (d+1)}$ (data with bias column), $Y \in \mathbb{R}^{n \times c}$, $\Theta \in \mathbb{R}^{(d+1) \times c}$.

The normal equations associated with this problem are $(X^\top X + n\lambda I)\Theta = X^\top Y$; However, computing the QR factorization of $X^\top X + n\lambda I$ would require explicitly forming $X^\top X$, negating the numerical advantage of the QR factorization discussed in Section 3.1.2 (forming $X^\top X$ squares the condition number of the problem).

To avoid this, a standard equivalence is exploited: defining the augmented matrix and vector

$$\tilde{X} = \begin{bmatrix} X \\ \sqrt{n\lambda} I_{d+1} \end{bmatrix} \in \mathbb{R}^{(n+d+1) \times (d+1)}, \quad \tilde{Y} = \begin{bmatrix} Y \\ 0 \end{bmatrix} \in \mathbb{R}^{(n+d+1) \times c}, \quad (1.24)$$

it occurs that

$$\|\tilde{X}\Theta - \tilde{Y}\|_F^2 = \|X\Theta - Y\|_F^2 + n\lambda \|\Theta\|_F^2, \quad (1.25)$$

That is, the regularized problem (Eq. 1.23) is equivalent to an ordinary (non-regularized) least squares problem on the augmented matrix $\tilde{X}$. The QR factorization is therefore applied to $\tilde{X}$, not to X alone and not to $X^\top X$.

1.4.1 Factorization and Resolution

The overall procedure is summarized in Algorithm 1: the ridge regression problem is first reduced to an ordinary least squares problem on the augmented matrix $\tilde{X}$, which is then solved via Thin QR factorization (Algorithm 2, detailed in Sec. 1.4.2) followed by backward substitution.

The Thin QR factorization of $\tilde{X}$ is computed:

$$\tilde{X} = Q_1 R_1, \quad Q_1 \in \mathbb{R}^{(n+d+1) \times (d+1)}, \quad R_1 \in \mathbb{R}^{(d+1) \times (d+1)}, \quad (1.26)$$

via Householder reflectors. The Thin variant is preferred to the Full one because, since $n \gg d$, the portion of Q beyond the first $d + 1$ columns does not contribute to the solution: calculating it would entail unnecessary computational and memory costs, with no benefit.

By exploiting the orthogonality of Q_1 (which preserves the norms), the least squares problem reduces to a triangular system:

$$\|\tilde{X}\Theta - \tilde{Y}\|_F^2 = \|R_1\Theta - Q_1^\top \tilde{Y}\|_F^2, \quad (1.27)$$

which, since R_1 is upper triangular and non-singular (guaranteed by regularization, cf. Sec. 3.1.2), is solved exactly by *backward substitution*:

$$R_1\Theta = Q_1^\top \tilde{Y}. \quad (1.28)$$

No explicit matrix inversion is ever computed.

1.4.2 Householder QR factorization implementation

The Householder QR factorization was implemented from scratch, without relying on library routines such as `numpy.linalg.qr`, in compliance with the project requirement of not using off-the-shelf solvers. The implementation follows a recursive column-elimination scheme, described in Algorithm 2.

Given a matrix $A \in \mathbb{R}^{\tilde{n} \times p}$ (in our call, $A = \tilde{X}$, with $\tilde{n} = n + d + 1$ rows and $p = d + 1$ columns), the algorithm eliminates one column at a time via a sequence of Householder reflectors, implemented recursively on progressively smaller submatrices.

At each step, the algorithm operates on the current leading column x of the active submatrix:

- **Reflector construction:** A scalar $\iota = -\text{sign}(x_1) \|x\|_2$ is computed, with the sign convention chosen to avoid catastrophic cancellation when computing $x - \iota e_1$ (if $x_1 = 0$, ι is set to $\|x\|_2$ to avoid a degenerate reflector). The Householder vector is then defined as $v = x - \iota e_1$ and normalized to unit norm, $v \leftarrow v / \|v\|_2$; if $\|v\|_2$ is below a numerical tolerance (10^{-12}), v is set to zero (the column is already aligned with e_1 and no reflection is needed).
- **Reflector application:** Since v is unit-norm, the Householder reflector takes the simplified form $H = I - 2vv^\top$, and is applied to the whole active submatrix as $A \leftarrow A - 2v(v^\top A)$, zeroing out all sub-diagonal entries of the first column.
- **Recursion:** The algorithm recurses on the trailing submatrix (rows and columns 2 : *end*) obtained after the reflection, following the same procedure until the submatrix has zero columns (base case).

- **Reconstruction:** On the way back from the recursion, the upper-triangular factor R is reassembled block-wise from the transformed first row/column at each level and the R returned by the recursive call. The orthogonal factor Q is built consistently: at each level, the identity block of appropriate size is reflected via the same Householder vector used to eliminate that column, and combined with the Q returned by the recursive call, so that Q is accumulated explicitly rather than left implicit.

Algorithm 1 Ridge Regression via Thin QR

Require: $X \in \mathbb{R}^{n \times (d+1)}$, $Y \in \mathbb{R}^{n \times c}$, $\lambda > 0$

- 1: $\tilde{X} \leftarrow [X; \sqrt{n\lambda} I_{d+1}]$ ▷ augmented matrix, $(n + d + 1) \times (d + 1)$
 - 2: $\tilde{Y} \leftarrow [Y; 0_{(d+1) \times c}]$
 - 3: $(Q_1, R_1) \leftarrow \text{THINQR}(\tilde{X})$ ▷ via Householder reflectors
 - 4: $Z \leftarrow Q_1^\top \tilde{Y}$
 - 5: $\Theta \leftarrow \text{BACKWARDSUBSTITUTION}(R_1, Z)$ ▷ solves $R_1 \Theta = Q_1^\top \tilde{Y}$
 - 6: **return** Θ
-

1.4.3 Computational cost

The Thin QR factorization of a matrix $(n + d + 1) \times (d + 1)$ via Householder reflectors with $n \gg d$ requires about $2nd^2$, and therefore we can define the cost of the order of $O(nd^2)$. The subsequent resolution of the triangular system by backward substitution costs $O(d^2)$ for each of the c output columns, hence $O(cd^2)$, negligible compared to the cost of factorization. The overall cost of the algorithm is therefore $O(nd^2)$.

Note that the recursive formulation implemented (Algorithm 2) explicitly forms and returns the orthogonal factor Q at every recursion level, rather than only applying the reflectors to $\tilde{Y}$ lazily. While this is not the most memory-efficient variant of Householder QR (which would typically store only the reflector vectors and apply them on demand), the cost remains dominated by the same $O(nd^2)$ term for $n \gg d$, since $p = d + 1 \ll \tilde{n} = n + d + 1$ and the recursion depth is bounded by p .

Algorithm 2 Recursive Thin QR via Householder Reflectors

```

1: function THINQR( $A \in \mathbb{R}^{\tilde{n} \times p}$ )
2:   if  $p = 0$  then
3:     return  $Q \leftarrow \emptyset_{\tilde{n} \times 0}, R \leftarrow \emptyset$ 
4:   end if
5:    $x \leftarrow A[:, 1]$ 
6:    $\iota \leftarrow -\text{sign}(x_1) \cdot \|x\|_2$ 
7:   if  $\iota = 0$  then
8:      $\iota \leftarrow \|x\|_2$  ▷ avoid degenerate reflector when  $x_1 = 0$ 
9:   end if
10:   $v \leftarrow x - \iota e_1$ 
11:  if  $\|v\|_2 > \text{tol}$  then
12:     $v \leftarrow v / \|v\|_2$ 
13:  else
14:     $v \leftarrow 0$ 
15:  end if
16:   $A \leftarrow A - 2v(v^\top A)$  ▷ apply reflector to the whole block
17:   $(Q_{\text{sub}}, R_{\text{sub}}) \leftarrow \text{THINQR}(A[2:\tilde{n}, 2:p])$  ▷ recurse on trailing submatrix
18:   $R \leftarrow \begin{bmatrix} A_{1,1} & A_{1,2:p} \\ 0 & R_{\text{sub}} \end{bmatrix}$ 
19:   $Q_{\text{block}} \leftarrow \begin{bmatrix} 1 & 0 \\ 0 & Q_{\text{sub}} \end{bmatrix}$  ▷ size  $\tilde{n} \times p$  identity-like block
20:   $Q \leftarrow Q_{\text{block}} - 2v(v^\top Q_{\text{block}})$  ▷ reflect the block to accumulate  $Q$ 
21:  return  $Q, R$ 
22: end function

```

Chapter 2

Expected results

2.1 Empirical estimation of the optimal value

Since no closed-form solution exists for the global optimum of the non-convex objective function L_{M1} , all analyses regarding convergence speed rely on an estimation of the optimal value f_λ^* . To establish a robust empirical baseline, this value was approximated by running all the algorithms, configured with their optimal hyperparameters, for an extended duration, incrementally ten times the length of a standard training run (20.000 epochs). The estimation is calculated separately for each regularization coefficient λ as the absolute minimum value observed across all these incrementally extended runs. Therefore, the value is defined as follows:

$$\bar{f}_\lambda = \min_{\text{run}, k} L_{M1, \lambda}(\theta_k). \quad (2.1)$$

The minimum value recorded up to any given iteration satisfies the relation $\bar{f}_i \geq f_\lambda^*$, given that no value observed during optimization can be lower than the true optimum. Consequently, the gap reported in this document,

$$r_k = L_{M1, \lambda}(\theta_k) - \bar{f}_\lambda, \quad (2.2)$$

constitutes a valid *lower bound* for the actual optimization gap $L_{M1, \lambda}(\theta_k) - f_\lambda^*$; indeed, the condition $\bar{f}_\lambda \geq f_\lambda^*$ implies that $r_k \leq L_{M1, \lambda}(\theta_k) - f_\lambda^*$. Any convergence rate estimated based on r_k must therefore be interpreted as an optimistic bound.

This limitation is inherent to methods based on local optima; there is no reasonable stopping criterion to certify how close $\bar{f}_i$ is to f^* at any finite iteration. Since an f_λ^* is not available—unlike in Lagrangian relaxation [1]— $\bar{f}_\lambda$ serves as the best available reference; the claim is not that it is numerically close to f_λ^* , but rather that it provides a valid bound and improves monotonically as the number of observed runs and iterations increases.

A separate estimate of $\bar{f}_\lambda$ is required for each value of λ , rather than a single global proxy, because λ is part of the definition of the objective function itself: $L_{M1, \lambda}(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|_1$ describes a different optimization problem for each λ , characterized by its own cost function

topology and, generally, its own optimal value f_λ^* . Estimating $\bar{f}_\lambda$ for each λ ensures that the convergence gap reflects, for each problem, the speed at which the algorithm approaches its specific optimum.

2.2 Algorithm A1

Theoretical convergence and Hypothesis verification

The convergence theory relies on the following assumptions:

- **Convexity:** Classical convergence results for the Heavy Ball method assume that the objective function is strongly convex and continuously differentiable. Under these assumptions, appropriate choices of learning rate and momentum coefficient guarantee linear convergence to the unique global optimum. In our case, however, the objective function is non-convex due to the non-linear hidden layer and the Leaky ReLU activation function. Although the $L1$ regularization term is convex, the overall optimization problem remains non-convex. Consequently, the classical convergence guarantees of the Heavy Ball method no longer hold.
- **Differentiability:** Heavy Ball is traditionally analyzed for smooth objective functions. Leaky ReLU activation is differentiable almost everywhere, unlike the standard ReLU, but the $L1$ regularization introduces non-differentiability whenever a parameter is equal to zero. In practice, optimization proceeds by employing a valid subgradient of the regularization term, allowing the Heavy Ball iterations to remain well defined.

For completeness, the theoretical convergence rate of Heavy Ball can be expressed explicitly in terms of the smoothness constant S and the strong convexity constant μ . For an S -smooth and μ -strongly convex objective, with condition number $\kappa = L/\mu$, the Heavy Ball method admits the parameter choices:

$$\eta = \frac{4}{(\sqrt{S} + \sqrt{\mu})^2}, \quad \alpha = \left(\frac{\sqrt{S} - \sqrt{\mu}}{\sqrt{S} + \sqrt{\mu}} \right)^2, \quad (2.3)$$

which lead to the linear convergence factor:

$$r = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}. \quad (2.4)$$

Thus, the distance from the optimum behaves approximately as $d_i \approx \leq r^i d_0$, giving an iteration complexity of:

$$O \left(\sqrt{\kappa} \log \frac{1}{\varepsilon} \right) \quad (2.5)$$

to reach an error of ε (the empirical counterpart of ε used throughout this report, in the absence of a closed-form f^* , is discussed in Sec. 2.1). This improves, in the strongly convex setting, upon the $O(\kappa \log(1/\varepsilon))$ dependence associated with standard Gradient Descent. These results are not directly applicable to the non-convex objective considered here, but they provide a useful theoretical reference for understanding the expected efficiency of the Heavy Ball method. In particular, they show explicitly how the condition number and the choice of momentum and learning rate can influence the convergence speed. [9, Theorem 9, point 3, p.13]

Since the assumptions of strong convexity and global differentiability are violated, there is no formal guarantee that the algorithm converges to the global optimum. More generally, momentum-based first-order methods applied to non-convex optimization are expected to converge, under suitable conditions, to first-order stationary points [4, Cor.(4.1–4.2), p. 168] under the additional assumption that the objective satisfies the Kurdyka–Łojasiewicz property. This applies to our setting since L_{M1} , being a finite sum of semi-algebraic (Leaky ReLU, $L1$), satisfies the KL property almost everywhere [4, Sec.2, p. 160]. Therefore, the algorithm is expected to stabilize around a local minimum, saddle point, or flat region of the loss landscape rather than the global optimum.

From a practical perspective, Heavy Ball momentum is widely employed, despite the lack of global convergence guarantees. Empirically, it consistently converges toward local minima that often exhibit good generalization performance.

Speed and Monotonicity

For strongly convex quadratic objectives, the Heavy Ball method enjoys an accelerated linear convergence rate with respect to standard Gradient Descent, reducing the dependence on the condition number from approximately κ to $\sqrt{\kappa}$ [9, Theorem 9, point 3, p.13]. These theoretical acceleration results, however, do not extend to the non-convex optimization problem considered here.

Therefore, we expect the following qualitative behavior:

- **Non-monotonicity:** Because of the inertial contribution introduced by the momentum term, the objective function may temporarily increase between consecutive iterations. Consequently, the training loss is expected to exhibit a non-monotonic behavior with small oscillations, particularly during the initial stages of training or when the momentum coefficient is relatively large.
- **Effect of Momentum:** The momentum term accumulates information from previous gradients, accelerating the optimization along directions of consistent descent while damping oscillations across narrow valleys. Compared with standard Gradient Descent, it has been empirically observed to produce smoother optimization trajectories and significantly reduce the epochs required to reach a stable solution.

Complexity and Efficiency

From a computational perspective, the cost of each iteration is dominated by two factors:

1. Computing the gradient of the objective function through backpropagation requires a number of operations linearly proportional to the number of trainable parameters, yielding a computational complexity of $O(t_\theta)$ for each processed sample.
2. Updating the momentum vector requires only vector additions and scalar multiplications. Both operations have linear complexity, namely $O(t_\theta)$.

Consequently, the total computational complexity of each iteration is identical to that of standard Gradient Descent, namely $O(t_\theta)$, while requiring only an additional memory cost of $O(t_\theta)$ to store the momentum vector.

Overall, the computational overhead introduced by Heavy Ball momentum is of the same asymptotic order as the gradient computation. In practice, the negligible amount of additional memory required is largely compensated by the reduced number of epochs typically required to obtain convergence, potentially making *A1* more computationally efficient than standard Gradient Descent in terms of the total number of iterations required to reach a stable solution.

2.3 Algorithm A2

Theoretical convergence and Hypothesis verification

The *A2* algorithm implements a deflected subgradient method to optimize the non-smooth and non-convex objective function of the *M1* model.

The subgradient theory is based on the following properties:

- **Convexity:** Standard convergence theorems for subgradient methods assume convexity of the objective function to ensure global optimality. In our case, although the regularization term *L1* is convex, the loss of *M1* is non-convex due to the presence of the nonlinear hidden layers of the neural network, so there are no guarantees of convergence to the global optimum.
- **Differentiability:** The objective function is not differentiable everywhere due to the regularization term *L1*. This makes classical gradient convergence theorems inapplicable and justifies the use of subgradients.

"Closest" theoretical results available: In the absence of convexity, the mathematical literature guarantees asymptotic convergence of the subgradient only towards a **stationary Clarke point** (a point where $0 \in \partial L_{M1}(\theta)$), i.e., a local minimum, a maximum, or a saddle point [6].

We expect that the algorithm will not find the global optimum, but will stabilize at a local minimum or plateau, as introduced in Sec. 2.1. However, in machine learning, local minima of well-configured neural networks often offer excellent generalization performance.

Speed and Monotonicity

Subgradient methods applied to convex problems exhibit a convergence rate typically equal to $O(1/\sqrt{k})$ [10, Theorem 3.2, p.264-265]. This speed can be defined as *slow* with respect to gradient methods.

In the non-convex scenario, we expect a trend characterized by:

- **Non-monotonicity:** In fact, the subgradient method is not a descent method; the direction of the subgradient is not necessarily a local descent direction. As a result, the loss on the training set will show an oscillatory trend.
- **Effect of Deflection and Restrictions:** Deflection (via *Volume Algorithm*) and the application of the *Stepsize-restricted* rule are intended to dampen these oscillations. We expect this restriction to help stabilize the loss in the final stages of training.

Indeed, in the results obtained, the techniques introduced limited the oscillations, ensuring smooth and stable curves.

Complexity and Efficiency

From a computational point of view, the cost per iteration of A2 is dominated by two components:

1. Calculating the subgradient g_i using the *backpropagation* algorithm, which has a time complexity linear with respect to the number of network parameters, $O(t_\theta)$ for each sample of the dataset.
2. The resolution of the deflection rule. In the case of the *Volume Algorithm*, the calculation of the parameter γ_i^* is done via a closed-form formula (Eq. 1.19) based on scalar products and vector norms in $\mathbb{R}^{t_\theta}$. The cost of this operation is strictly linear, $O(t_\theta)$.

Consequently, the single-iteration computational efficiency of A2 is comparable to that of a standard gradient method, requiring only a minimal amount of additional memory to store the previous direction d_{i-1} and the linearization error. The real challenge in terms of overall efficiency lies in the large number of epochs required to achieve convergence due to the sublinear convergence rate.

2.4 Algorithm A3

Theoretical guarantees and Hypothesis verification

Algorithm A3 is a direct (non-iterative) method applied to solve the ridge regression problem associated with model $M2$.

From a theoretical point of view, the QR (Thin) factorization applied to the augmented matrix $\tilde{X}$ (Eq. 1.24) requires that $\tilde{X}$ have full column rank, so that the triangular matrix R_1 is non-singular and the solution of the system $R_1\Theta = Q_1^\top \tilde{Y}$ is unique. Since $\tilde{X}$ contains by construction

the block $\sqrt{n\lambda} I_{d+1}$, for every $\lambda > 0$ the matrix $\tilde{X}$ has full column rank regardless of the rank of X and the presence of collinearity between the features (cf. Sec. 3.1.2): the assumptions of the method are therefore always satisfied in our problem.

Stability and quality of the approximation

Being a non-iterative algorithm, there are no problems of asymptotic convergence speed or non-monotonicity.

From the perspective of numerical stability:

- **Stability of QR factorization:** Householder reflector factorization enjoys the property of *backward stability* [11, Ch. 3]: the numerically computed matrix Q_1 preserves orthogonality within machine precision ($Q_1^\top Q_1 \approx I$).
- **Comparison with normal equations:** By operating on the augmented matrix $\tilde{X}$ rather than $X^\top X + n\lambda I$, we avoid squaring the condition number discussed in Section 3.1.2 ($\kappa(X^\top X) = \kappa(X)^2$), guaranteeing a solution whose sensitivity to data perturbations remains tied to $\kappa(\tilde{X})$ rather than $\kappa(\tilde{X})^2$.

The algorithm is therefore expected to return a numerically robust and stable solution in a single computational step, without oscillations or divergences.

Computational cost and comparison with Alternative methods

The Thin QR factorization of the augmented matrix $\tilde{X} \in \mathbb{R}^{(n+d+1) \times (d+1)}$ has a cost of $O(nd^2)$ for $n \gg d$, which is dominant compared to the subsequent resolution of the triangular system, which costs $O(cd^2)$.

A natural comparison is with the SVD factorization, which for a matrix of the same size would require $O(nd^2 + d^3)$ FLOPs. In our problem $d \ll n$ (13 columns versus hundreds of samples), for which the $O(d^3)$ term is dominated by $O(nd^2)$: **QR and SVD therefore have the same order of asymptotic complexity** in this regime.

The preference for QR is instead justified by two practical considerations:

1. **Multiplicative constant:** For the same order of complexity, the QR factorization requires a single triangularization step via Householder reflectors, while the SVD calculation typically requires a bidiagonalization followed by an iterative process for calculating singular values, resulting in a significantly higher multiplicative constant hidden in the $O(\cdot)$ notation.
2. **Information not necessary for the problem:** the SVD provides additional features (rank diagnosis, singular values for conditioning analysis, compression capabilities) that are not necessary in our case: the full rank of $\tilde{X}$ is already analytically guaranteed by the regularization for every $\lambda > 0$, and the conditioning analysis was conducted separately in the exploratory phase (Sect. 3.1.2).

We therefore conclude that QR is the most efficient choice in practice, given the same stability guarantees actually required by the problem, although it does not enjoy an asymptotic advantage over SVD in our specific dimensional regime.

Finally, we expect the execution of $A3$ to be nearly instantaneous compared to the iterative training times required by $A1$ and $A2$.

Chapter 3

Experimental set-up

3.1 Dataset

The dataset used for the evaluation of the models is ML-CUP25-TR, provided as part of the *Machine Learning* course. It is a multivariate regression problem, whose main characteristics are summarized below:

- **Number of samples:** $n = 500$
- **Input dimension:** $d = 12$, each sample has 12 features: $x_i \in \mathbb{R}^{12}$
- **Output dimension:** $c = 4$, the target: $y_i \in \mathbb{R}^4$

3.1.1 Pre-processing

The input features were standardized using *z-score*, computing the mean and standard deviation. The same procedure was also applied to the target variables, given the multi-output nature of the problem, to prevent components of y with a larger range from dominating the loss function. Standardization is motivated both by practical reasons for the iterative algorithms *A1* and *A2*, whose convergence speed depends on the relative curvature of the objective function along the different parameter directions, and by a quantifiable benefit for the direct algorithm *A3*: as shown in Table 3.1, normalization reduces the condition number of X from 86.06 to 32.19, improving the numerical stability of the solution.

Indeed, gradient-based descent algorithms are sensitive to the scale of the features. If the features have very different ranges, the loss surface has very different curvatures along the different directions, which slows down convergence and makes it more difficult to choose a single learning rate η suitable for all directions simultaneously.

3.1.2 Collinearity and conditioning

The dataset exhibits strong collinearity among the features: the correlation matrix shows coefficients typically ranging between 0.97 and 0.99 in absolute value for most pairs of features, with the exception of feature 6, which is relatively less correlated with the others ($|corr| \leq 0.80$). Despite this, the condition number of X is relatively contained (Table 3.1, $\kappa(X) \approx 32.19$ after normalization, compared to 86.06 on the raw data), indicating that the regression problem, despite presenting redundancy among the features, is not severely ill-conditioned from a numerical perspective. The numerical verification also confirms the theoretical relationship

$$\kappa(X^\top X) = \kappa(X)^2 (1036.1 \approx 32.19^2) \quad (3.1)$$

justifying the choice to avoid explicitly forming $X^\top X$.

Quantity	Value
$\kappa(X)$ (raw data)	8.61×10^1
$\kappa(X)$ (normalized)	3.22×10^1
$\kappa(X^\top X)$	1.04×10^3
$\kappa(X)^2$ (theoretical verification)	1.04×10^3

Table 3.1: Condition number of X and $X^\top X$, with verification of the theoretical relationship $\kappa(X^\top X) = \kappa(X)^2$

The introduction of ridge regularization further improves the conditioning: for $\lambda = 1$ we have $\kappa(X^\top X + n\lambda I) \approx 12.3$, almost two orders of magnitude lower than in the non-regularized case ($\lambda = 0$) (Table 3.2).

λ	$\kappa(X^\top X + n\lambda I)$
0	1.036×10^3
0.0001	1.027×10^3
0.001	9.502×10^2
0.01	5.444×10^2
0.1	1.040×10^2
1	1.232×10^1

Table 3.2: Effect of regularization on the conditioning of the normal equations system

3.2 Model Selection

3.2.1 Model selection for M1

The architecture of $M1$ (number of hidden neurons h_1, h_2 and activation function) was fixed to a single reasonable configuration, without performing a validation-based search over these hyperparameters.

The chosen configuration is $(h_1^*, h_2^*, \text{act}^*) = (70, 50, \text{leakyReLU})$: a network of this size yields a parameter vector $\theta \in \mathbb{R}^{t_\theta}$ with $t_\theta = 4664$ components¹, ensuring that the resulting optimization problem L_{M1} is genuinely high-dimensional and non-convex, without being so large as to make an extensive comparison across algorithms and regularization values computationally prohibitive. This configuration is fixed and used identically throughout the rest of the report, for both $A1$ and $A2$, so as to ensure that both algorithms solve exactly the same optimization problem.

Regarding weight initialization, a single realization of He initialization (appropriate for *leakyReLU*) was fixed via a random seed and reused identically across all subsequent experiments (varying λ , algorithm, and stepsize rule), so that differences in convergence behavior can be attributed to the algorithms and to λ rather than to initialization variability.

Regularization coefficient λ , unlike the architectural hyperparameters above, was *not* fixed to a single value: since λ directly affects the loss landscape traversed by $A1$ and $A2$, its effect on convergence speed is the central object of study of this report, and is examined systematically in Sec. 4 across the decades $\lambda \in \{1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7\}$.

3.2.2 Model selection for M2

The only hyperparameter for $M2$ is the ridge regularization coefficient λ , since Algorithm $A3$ is a direct method and has no algorithmic parameters of its own.

The optimal value of this hyperparameter is $\lambda^* = 0.1$, as a compromise between numerical conditioning and data fidelity. From the conditioning analysis in Section 3.1.2, for $\lambda^* = 0.1$ we obtain $\kappa(X^\top X + n\lambda^*I) \approx 104.0$ (Table 3.2), an improvement of almost an order of magnitude compared to the unregularized case ($\lambda = 0$). At the same time, the training MSE increases by only about 2% compared to $\lambda = 0$ (Table 4.11), while larger values of λ worsen the solution value.

This configuration is fixed and used unchanged in all experiments on the $A3$ algorithm.

3.3 Algorithms Hyperparameters Selection

As previously introduced, two stepsize rules were tested for the $M1$ model: ColorTV and SGPTL.

Selection criterion for subgradient algorithm hyperparameters The selection of hyperparameters evaluates three metrics simultaneously on the training loss $L_{M1,\lambda}$ at the reference

¹ $t_\theta = (12 + 1) \cdot 70 + (70 + 1) \cdot 50 + (50 + 1) \cdot 4 = 4664$, accounting for weights and biases of all three layers.

value $\lambda = 10^{-3}$: the final training loss attained, L_{final} ; the training *oscillation*, defined as the fraction of training epochs in which the loss increases with respect to the previous epoch; and the wall-clock time required to reach convergence, T .

Configurations resulting in loss divergence are excluded *a priori*, as no meaningful score can be assigned to them. For all remaining configurations, each metric is independently min-max normalized across the candidate configurations,

$$\hat{z} = \frac{z - z_{\min}}{z_{\max} - z_{\min}} \in [0, 1], \quad z \in \{L_{\text{final}}, \text{osc}, T\}, \quad (3.2)$$

so that $\hat{z} = 0$ corresponds to the best-performing configuration on that metric and $\hat{z} = 1$ to the worst. The three normalized metrics are then combined into a single composite score,

$$\text{score} = w_L \hat{L}_{\text{final}} + w_{\text{osc}} \widehat{\text{osc}} + w_T \hat{T}, \quad (3.3)$$

with weights $w_L = w_{\text{osc}} = w_T = 1/3$, reflecting equal a priori importance given to solution quality, stability, and computational efficiency. The configuration minimizing Eq. (3.3) is selected.

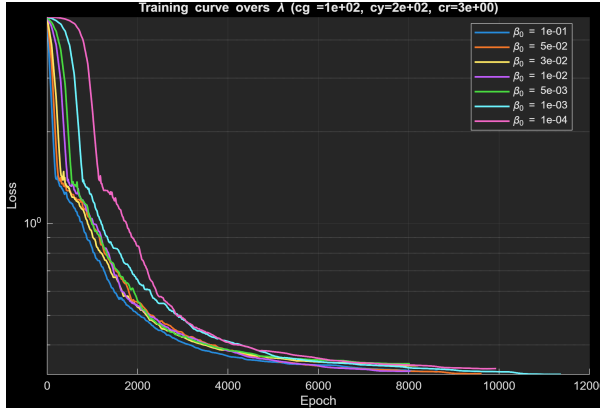
3.3.1 Stepsize rules hyperparameters

ColorTV Rule Hyperparameters The parameters required to define the ColorTV Rule are:

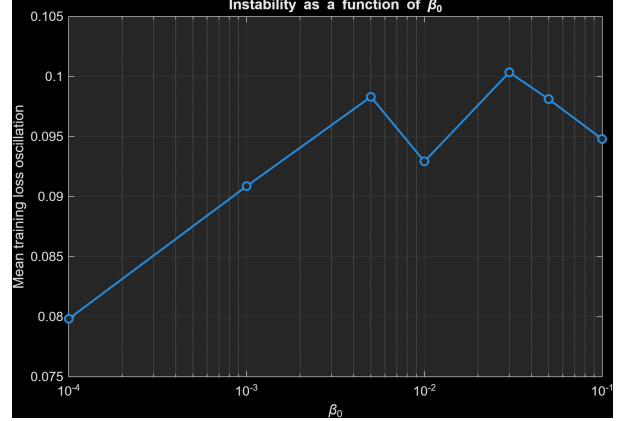
- β_0 : initial value of the scaling factor β of the stepsize rule.

Tested values: $\beta_0 \in \{1e-1, 5e-2, 3e-2, 1e-2, 5e-3, 1e-3, 1e-4\}$

This parameter directly controls the trade-off between convergence speed and stability. Larger values of β_0 consistently yield faster convergence: in Fig. 3.1a the configuration $\beta_0 = 1e-1$ reaches the training loss plateau in roughly 8000 epochs, compared to over 10000 epochs for $\beta_0 = 1e-4$. In the deflection setting, this acceleration does not come at the cost of a proportionally larger instability: the mean oscillation of the training loss (Fig. 3.1b) remains confined to a narrow range (0.08–0.10) across the whole tested interval. Conversely, when using Heavy Ball, excessively high values of β_0 cause significant instability in the algorithm, even though it continues to converge faster.



(a) Training curves with different β_0 values



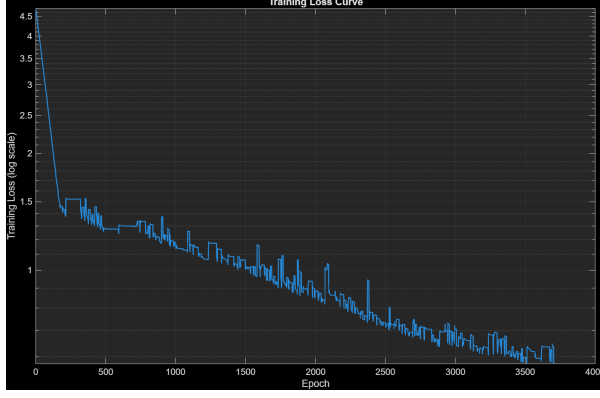
(b) Mean oscillation based on β_0 values

Figure 3.1: Loss curves based on β hyperparameter for ColorTV Stepsize Rule (implementation with deflection)

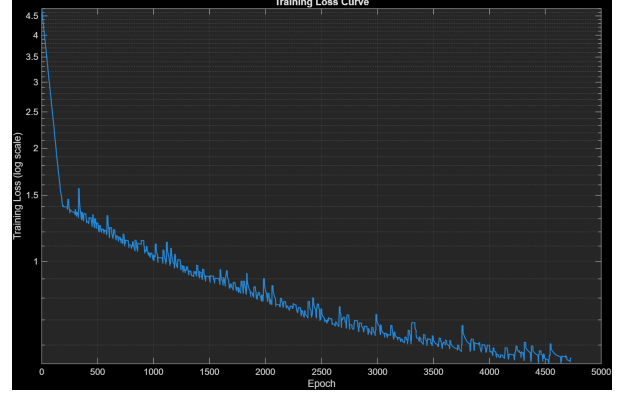
Based on the composite score of Eq. (3.3) (combining final training loss, oscillation, and convergence time):

- $\beta_0 = 1e - 1$ was selected for deflection setting, as it achieves the fastest convergence while remaining among the configurations with the lowest oscillation.
- $\beta_0 = 1e - 2$ was selected for momentum setting to reduce oscillations while attempting to maintain rapid convergence.
- c_g, c_y, c_r : thresholds on the number of consecutive green/yellow/red iterations before updating β .
 - $c_g \in \{30, 50, 70, 100, 150\}$
 - $c_y \in \{200, 400\}$
 - $c_r \in \{3, 5, 10, 20\}$

The grid search revealed that the selection of values for c_g and c_r was indeed significant for stability and convergence, whereas the c_y hyperparameter played a marginal role. In deflection setting, to ensure stable convergence, the c_g parameter value must be ≥ 100 , while the c_r hyperparameter must be ≤ 5 (examples in Fig 3.2). For the Heavy Ball method, on the other hand, the optimal setting was identified as $c_g = 50$ and $c_r = 10$.



(a) Loss training curve with $cg = 70$



(b) Loss training curve with $cr = 20$

Figure 3.2: Examples of bad cr and cg hyperparameters values that lead to instability and strong oscillation

- ρ_{CT} : tolerance used to classify the type of iteration. Following the approach adopted in [1], this value was fixed at $1e - 6$.

Optimal ColorTV configuration The final optimal configurations selected for ColorTV step-size rule, based on selection rules in Sec. 3.3, are:

Heavy Ball: $(\beta_0^*, c_g^*, c_y^*, c_r^*, \rho_{CT}) = (1e - 2, 50, 200, 10, 1e - 6)$;

Volume Algorithm: $(\beta_0^*, c_g^*, c_y^*, c_r^*, \rho_{CT}) = (1e - 1, 100, 200, 3, 1e - 6)$;



Figure 3.3: Loss curve for optimal hyperparameters configurations for ColorTV stepsize rule

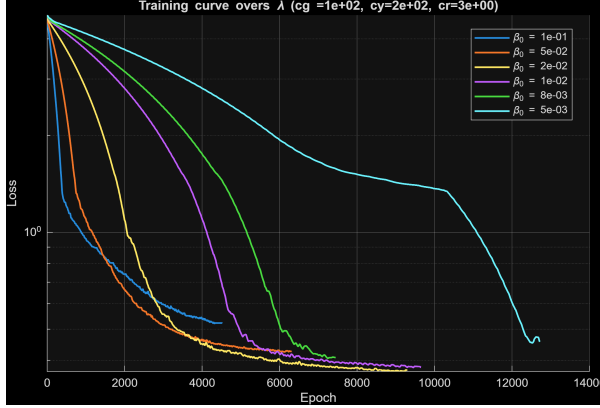
SGPTL Rule Hyperparameters The SGPTL rule was tested exclusively in combination with deflection; since it yielded poorer convergence results—in terms of values, oscillation, and speed—compared to the ColorTV rule (Sec. 4.1.2), it was not investigated further in combination with Heavy Ball.

The SGPTL rule requires the specification of the following parameters:

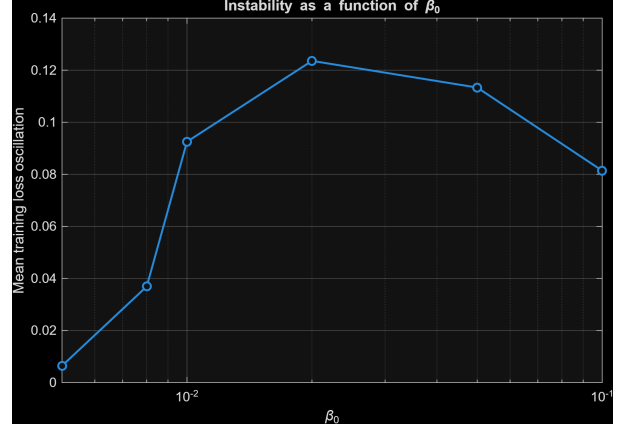
- β : scalar used in the computation of the deflected subgradient stepsize.

Tested values: $\beta \in \{1e-1, 5e-2, 1e-2, 8e-3, 5e-3\}$

As with the ColorTV rule, it plays a fundamental role in stability and convergence; higher values of β requires lower time but obtain higher loss values while values $\leq 1e-2$ lead to excessively slow convergence requiring a higher time of computation to obtain a similar loss value (figure 3.4).



(a) Loss curves with different β_0 values



(b) Mean oscillation based on β_0 values

Figure 3.4: Trade-off between lower loss and reduced oscillation for SGPTL Stepsize Rule

The configuration with $\beta = 0.02$, tested after a nested grid search, achieves a good trade-off between lower loss value, convergence rate number of training epochs.

- δ_0 : initial estimate of the gap from the optimal level f_{best} .

Tested values: $\delta_0 \in \{1, 0.8, 0.5, 0.3, 0.1, 0.07, 0.05\}$

This is an initialization hyperparameter; its value is not particularly important because, if suboptimal, it is dynamically updated by the algorithm. Despite this, the optimal value was identified at $\delta_0 = 0.5$, which allows for both a lower loss value and reduced oscillation.

- R : threshold on the accumulated path r beyond which δ is reduced.

Tested values: $R \in \{5e-1, 3e-1, 1e-1, 8e-2, 5e-2\}$

This hyperparameter plays a crucial role in determining after how many non-optimal steps the previous parameter should be updated. Identifying the optimal value is essential for ensuring the algorithm's stable convergence.

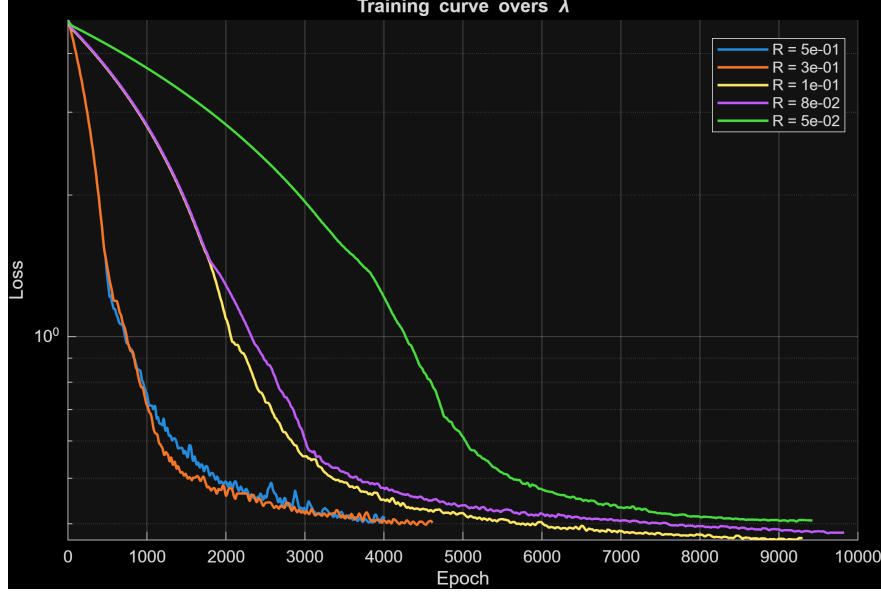


Figure 3.5: Curves of convergence based on R hyperparameter

Figure 3.5 shows that values of $R \geq 3e - 1$ lead to faster convergence but higher instability and higher final loss values; this is consistent with the role of R in the SGPTL rule. Conversely, values $R \leq 0.08$ make the loss descent too slow. For this reason, the value $R = 0.1$ was identified as the best trade-off between stability, convergence, and speed.

- $\rho_{SG} \in (0, 1)$: multiplicative factor used to reduce δ (not to be confused with the ColorTV tolerance ρ_{CT}).

Tested values: $\rho_{SG} \in \{0.7, 0.5, 0.3\}$

This update rate is linked to the δ_0 initialization value; lower values are preferable when δ_0 is large, and bigger values otherwise.

The values f_{ref} and f_{best} are initialized to $L(\theta_0)$, with $r = 0$.

Optimal SGPTL configuration The final optimal configuration selected for SGPTL stepsize rule, based on model selection rules in Sec. 3.3 is:

$$(\beta^*, \delta_0^*, R^*, \rho_{SG}) = (2e - 2, 5e - 1, 1e - 1, 5e - 1)$$

3.3.2 Algorithm A1: Heavy Ball

Since it is not possible to determine the optimal values of momentum α and learning rate η based on the assumptions and formulas presented previously, the Heavy Ball algorithm instead required the selection of a small set of hyperparameters, in addition to the hyperparameters of $M1$, using the Grid Search method, in order to identify the optimal ranges of values for each of them. This process was applied multiple times, narrowing the range of possible values at each

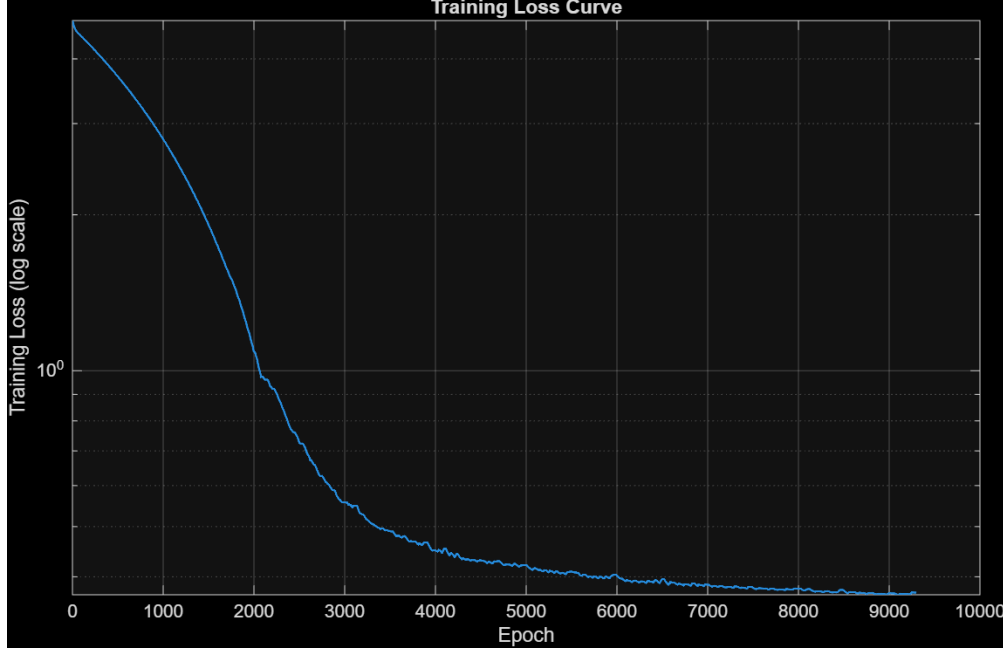


Figure 3.6: Loss curve for optimal hyperparameters configurations for SGPTL Stepsize Rule

iteration to the most promising ranges identified previously. Given the relatively small number of hyperparameters required, in this case it was possible to use a finer granularity in the selection of values without encountering a combinatorial explosion.

Heavy Ball Hyperparameters The Heavy Ball method requires the following parameters:

- *batch*: the number of training examples processed together before the algorithm updates its trainable parameters.

Tested values: $batch \in \{50, 100, 250, 500\}$

Although Heavy Ball is traditionally formulated using full-batch gradients, a mini-batch approach was also investigated to further explore the effect of the batch size on the algorithm's performance. The experiments showed that $batch = 250$ (half batch) can provide a slight improvement in performance, at the cost of increased instability in the curves. Since the observed improvement was marginal, the increase in instability was not considered worthwhile. Therefore, $batch = 500$ (full batch) was selected as the best compromise between convergence performance and curve stability.

- α : the coefficient controlling the strength of the momentum.

Tested values: $\alpha \in \{0.90, 0.85, 0.80, 0.75, 0.70, 0.60, 0.50, 0.30, 0\}$

Observed effects of momentum on convergence speed

The results obtained with the Heavy Ball method provide an experimental perspective on the convergence behavior discussed theoretically in Section 2.2. By evaluating the relative optimization gap, which normalizes the trajectory against the empirical optimum, we could precisely isolate the acceleration impact of the momentum coefficient α .

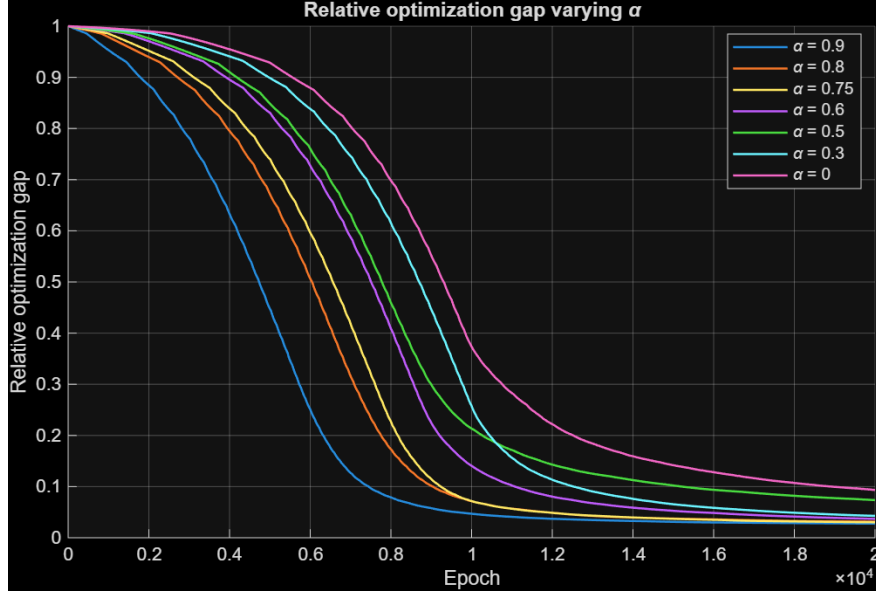


Figure 3.7: Relative optimization gap for different values of α .

As illustrated in Figure 3.7, different values of the momentum coefficient α were tested while keeping the remaining parameters fixed. The configuration with $\alpha = 0$ (pink curve), corresponding to standard Gradient Descent, decays considerably more slowly than all configurations using momentum, still leaving a relative gap of approximately 0.10 even after 20,000 epochs.

The speed of convergence improves drastically and monotonically as the momentum coefficient increases. The configuration with $\alpha = 0.9$ (dark blue curve) exhibits the steepest drop, closing the relative optimization gap in roughly half the epochs required by moderate values such as $\alpha = 0.5$ (green curve). It reaches a near-zero normalized gap around epoch 10,000, confirming that a high momentum value efficiently navigates the non-convex objective landscape to settle into the optimum without introducing severe instability. Based on this rapid and decisive gap reduction, $\alpha = 0.90$ is confirmed as the optimal choice for the A1 algorithm.

Observed effects of regularization on convergence speed

The choice of α must also be considered together with the regularization coefficient λ . While α directly controls the dynamics of the optimization algorithm, λ modifies the objective function itself by determining the relative importance of the L_1 penalty. Because each value of λ defines a distinct objective function with its own unique global optimum, comparing absolute training loss values across configurations is misleading. Instead, convergence speed must be evaluated using the relative optimization gap, which normalizes the trajectory against the empirical optimum $\bar{f}_\lambda$ specific to that exact regularization setting.

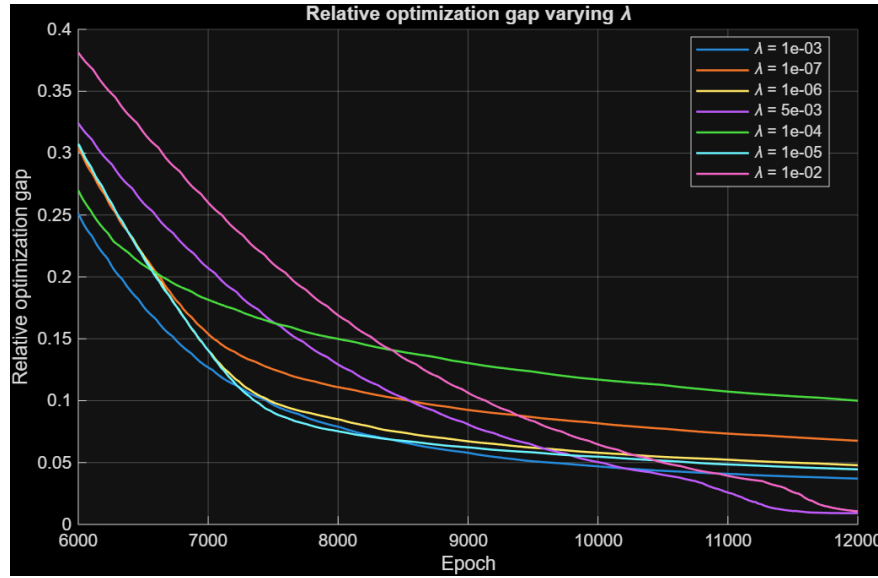


Figure 3.8: Relative optimization gap for different values of λ evaluated from epoch 6000 to 12000.

As illustrated in Figure 3.8, the regularization coefficient strictly dictates the algorithm’s ability to close the final optimization gap. The configuration with the strongest tested regularization, $\lambda = 10^{-2}$ (pink curve), exhibit a delayed drop in the relative gap. Although it maintain higher relative gaps early in the visualized window (epochs 6000-7000), it ultimately achieve the tightest convergence, reaching normalized gap values near 0.01 by epoch 12000. This suggests that the strong L_1 penalty tightly constrains the solution space, eventually allowing the Heavy Ball momentum to effectively settle into the specific optimum of this highly restricted landscape.

Conversely, reducing the regularization strength results in earlier stagnation relative to the respective optimal values. The configuration with $\lambda = 10^{-4}$ (green curve) flattens out noticeably higher than the others, struggling to reduce the relative gap below 0.10. Similarly, the lowest regularization values ($\lambda = 10^{-5}$ down to 10^{-7}) stabilize their decay earlier, leaving a residual relative gap between 0.05 and 0.07. In these less restricted, more complex loss landscapes, the standard Heavy Ball method exhibits difficulty performing the final, fine-grained refinements required to close the gap to the empirical minimum.

This observation is highly relevant for the choice of the final *M1* configuration. The value $\lambda = 10^{-3}$ (dark blue curve) provides a highly balanced convergence trajectory. It avoids the delayed convergence phase characteristic of the heavier penalties while achieving a tight, smooth decay to a relative gap of roughly 0.04—outperforming the asymptotic refinement of the lower λ values. Since the primary goal is a fair and equitable comparison between *A1* and *A2*, we selected a λ value that avoids both extreme spatial restriction and severe convergence stagnation. The final *M1* and *A1* configuration was therefore fixed at $(h_1, h_2, activation, \lambda, \alpha, batch) = (70, 50, leakyReLU, 10^{-3}, 0.90, 500)$ for the subsequent comparison with *A2*.

3.3.3 Algorithm A2: Volume algorithm

Volume Algorithm Hyperparameters The Volume Algorithm requires the specification of the following parameters, which are fundamental for the update of τ :

- τ_0 : initial value of the adaptive upper threshold for γ .
Tested values: $\tau_0 \in \{1, 0.5, 0.1\}$
This parameter allows the amount of deflection to be defined from the very first iterations. It has been observed that lower values — ideally 0.1 — lead to better convergence.
- τ_p : number of iterations between consecutive updates of τ .
Tested values: $\tau_p \in \{200, 100\}$
Lower values allow for a faster transition to strong deflection.
- $\tau_f \in (0, 1)$: decrement factor for τ .
Tested values: $\tau_f \in \{0.9, 0.7\}$
Both algorithms prefer a slow decrease $\tau_f = 0.9$.
- τ_{min} : minimum threshold for τ , fixed as $\tau_{min} = 1e - 5$
- m : threshold used to distinguish between Serious and Null steps.
Tested values: $m \in \{1e - 1, 1e - 2, 5e - 3\}$
It determines how much a step must improve the loss to be considered "significant." It has been observed that algorithms prefer more permissive values, such as $1e - 1$, which more readily accept a step as significant.

Optimal Volume Algorithm configuration The final selected hyperparameter for Volume Algorithm in combination of both the Stepsize Rules is:

$$(\tau_0^*, \tau_p^*, \tau_f^*, \tau_{min}^*, m^*) = (0.1, 200, 0.9, 1e - 5, 1e - 1);$$

The theoretical constraint $\beta_i \leq \gamma_i \leq 1$ is implemented, as described in Sec. 1.3.3, by setting $\beta_{eff} = \min(\beta, \gamma)$ when computing the stepsize η .

Chapter 4

Results and comments

4.1 Model M1: Heavy Ball vs Volume Algorithm

This section focuses on comparing the results, for the previously defined non-convex problem (Eq. 1.2), between the subgradient implementation using the Heavy Ball method, and the implementation using the deflection method — specifically, the Volume Algorithm.

For the comparison, the ColorTV step-size rule was selected; as demonstrated by the results in Sec. 4.1.2, it yielded superior performance in terms of convergence speed and values, as well as reduced oscillation.

To analyze the subgradient method, compare the two step-size rules (ColorTV and SGPTL), and compare the Volume Algorithm (deflection) with Heavy Ball (momentum), the algorithm used to calculate the slope for the various implementations is defined as follows.

Empirical estimation of the convergence rate The empirical decay of the best-known gap is analyzed using a power-law model:

$$r_k \propto k^s.$$

To mitigate the influence of oscillations and the initial settling phase, the analysis proceeds as follows:

1. **Monotonization of the cost function:** We consider the cumulative minimum of the cost curve,

$$L_k^{\text{best}} = \min_{j \leq k} L_j,$$

thereby eliminating upward oscillations from the convergence trajectory while preserving the best objective function value achieved up to iteration k .

2. **Construction of the best-known gap:** The reference value $\bar{f}_\lambda$, introduced in Sec. 2.1, is defined as the minimum objective function value observed across all runs and iterations

available for a given λ . The convergence curves analyzed in this section do not correspond to the specific runs that produced this minimum; they are selected from the best run in a smaller range of execution epochs. The empirical best-known gap is then calculated as

$$r_k = L_k^{\text{best}} - \bar{f}_\lambda.$$

Since $\bar{f}_\lambda$ is derived from a broader set of runs than the single trajectory under examination, it provides a common reference value for comparing convergence trajectories; the resulting gap should be interpreted as an empirical best-known gap rather than an exact distance from the unknown global optimum.

3. **Exclusion of the initial plateau:** The analysis is restricted to the observed decay regime, excluding the initial settling phase. In the computation of the slope, the fitting interval begins at $k_{\text{start}} = 100$ of the analyzed run, where an initial plateau is observed for all algorithms due to the initial settling phase.
4. **Log-log linear regression:** The logarithmic transformation

$$\log_{10}(r_k) = s \log_{10}(k) + c$$

is fitted to the data using least-squares linear regression within the selected interval. Points where $r_k \leq 0$ are excluded from the regression because the logarithm is undefined. The resulting slope s characterizes the empirical decay rate of the best-known gap within the chosen fitting interval.

The resulting slope measures the speed at which the analyzed trajectory reduces its gap relative to the best objective value observed for that λ . Since $\bar{f}_\lambda$ is an empirical reference rather than the unknown global optimum f_λ^* , the estimated slope should not be interpreted as a rigorous convergence rate toward the global optimum, but it provides a consistent empirical measure for comparing the decay of different optimization trajectories.

4.1.1 Estimation of the best-known value $\bar{f}_\lambda$

As introduced in Sec. 2.1 the convergence analyses use the best-known objective value $\bar{f}_\lambda$, estimated separately for each regularization coefficient λ , as defined in Eq. 2.1.

The lambda values that were tested are:

$$\lambda \in \{1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7\}$$

For $\lambda \geq 10^{-1}$, all three algorithms converge to a stable point, yet the resulting loss values are anomalously high compared to the rest of the range under consideration. Decomposing the objective function reveals that, at this order of magnitude, the L_1 penalty term numerically dominates

the loss and results are not interesting. Consequently, the primary convergence analysis is restricted to the range $\lambda \leq 10^{-2}$, where the regularization and data-fitting terms maintain a more balanced relationship.

λ	$L_{\text{ColorTV\&Volume}}$	$L_{\text{SGPTL\&Volume}}$	$L_{\text{HeavyBall}}$	$\bar{f}_\lambda = L_{\min,\lambda}$
$1e-7$	0.0022322	0.047837	0.055569	0.0022322
$1e-6$	0.0023357	0.059778	0.056356	0.0023357
$1e-5$	0.012385	0.06152	0.06206	0.012385
$1e-4$	0.067802	0.13201	0.12739	0.067802
$1e-3$	0.29102	0.34568	0.34023	0.29102
$1e-2$	0.67067	0.67178	0.69163	0.67067

Table 4.1: Best-known objective value $\bar{f}_\lambda = \min_{\text{run}, k} L_{M1,\lambda}(\theta_k)$ for each value of λ , obtained as the lowest training loss observed across all available runs and iterations.

From Table 4.1, we can make the following observations:

- The best-known objective value $\bar{f}_\lambda$ increases monotonically with λ , from approximately 0.0022 at $\lambda = 10^{-7}$ to approximately 0.671 at $\lambda = 10^{-2}$. This monotonic increase is consistent with what we expected. The increasing contribution of the L_1 regularization term to the objective function is observed as λ grows.
- The ColorTV-Volume configuration achieves the lowest observed objective value for all tested values of λ . The difference with the other two configurations is particularly pronounced for small regularization values, especially for $\lambda \leq 10^{-5}$, and becomes progressively smaller as λ increases. At $\lambda = 10^{-2}$, the three configurations reach very similar objective values.

4.1.2 Stepsize Rules: ColorTV vs SGPTL

λ	ColorTV		SGPTL	
	Loss	Osc.	Loss	Osc.
10^{-7}	0.0478	0.0190	0.1093	0.0719
10^{-6}	0.0363	0.0287	0.1072	0.0676
10^{-5}	0.0536	0.0295	0.1127	0.0705
10^{-4}	0.0949	0.0376	0.1829	0.0825
10^{-3}	0.3015	0.2090	0.3712	0.0759
10^{-2}	0.6810	0.2802	0.6951	0.1899

Table 4.2: Comparison between ColorTV and SGPTL across regularization values λ : final training loss and mean oscillation.

Table 4.2 illustrates the performance comparison between the ColorTV and SGPTL stepsize rules under identical model configurations and initial weights, with optimal algorithmic hyperparameters. The two stepsize rules are evaluated in terms of convergence values and oscillation behavior. All executions were performed with the standard maximum limit of 20000 iterations.

Loss convergence The results show that both methods achieve comparable performance, with ColorTV yielding consistently lower loss values across all tested values of λ . At $\lambda = 10^{-2}$, the performance gap narrows significantly, with both methods reaching similar values. These final values characterize the quality of the solutions reached by the two methods.

Oscillation behavior ColorTV demonstrates markedly superior oscillation control under low-to-medium regularization ($\lambda \leq 10^{-4}$). In this regime, ColorTV’s oscillation (0.0190 – 0.0376) is much lower than that of SGPTL ($\approx 0.068 - 0.083$), allowing for much smoother convergence trajectories. In addition, ColorTV exhibits a direct relationship between λ and oscillation: as the regularization strength increases, the optimization trajectory becomes progressively more unstable. In contrast, SGPTL maintains a relatively stable level of oscillation across lower values of λ , showing significant degradation only at $\lambda = 10^{-2}$.

Empirical convergence of the best-known gap To analyze the convergence speed, we consider the empirical best-known gap $r_k = L_k^{\text{best}} - \bar{f}_\lambda$. The decay of r_k is evaluated as described in Sec. 4.1. The resulting slopes (s) and goodness-of-fit (R^2) values are reported in Table 4.3, while Figure 4.1 illustrates the trajectory for $\lambda = 10^{-3}$ over the range $k \in [10^2, 2 \cdot 10^4]$ epochs. A more negative value of s indicates a steeper empirical decay over the fitted interval.

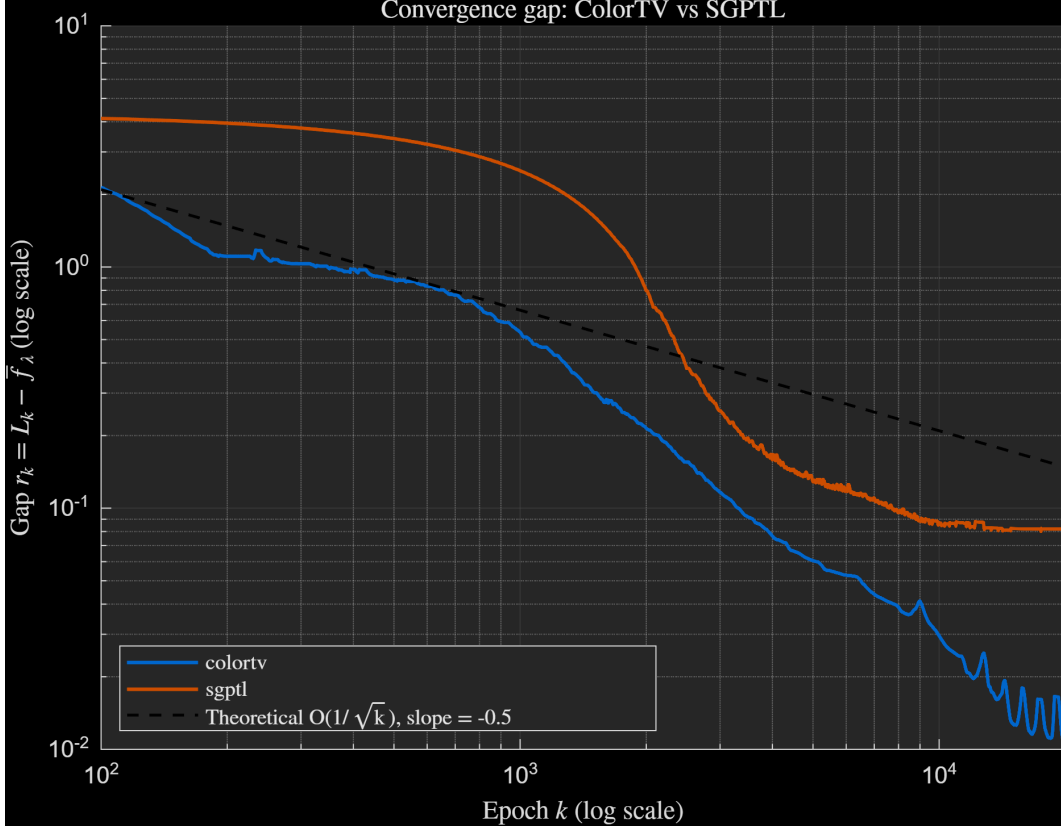


Figure 4.1: Direct comparison of the best-known gap trajectories between ColorTV and SGPTL ($\lambda = 10^{-3}$, M1 configuration, same weight initialization).

The slope results in Table 4.3 do not show a uniformly steeper decay for either stepsize rule: SGPTL exhibits more negative slopes for $\lambda \in [10^{-7}, 10^{-4}]$, whereas ColorTV achieves steeper slopes for $\lambda \in [10^{-3}, 10^{-2}]$.

Figure 4.1 allows us to make some important observations on the behavior of the gap trajectory. For $\lambda = 10^{-3}$ (but in general for all values of λ), SGPTL spends its first iterations in a high-gap plateau, then reduce the gap with a steep descent in subsequent iterations and finally reach a new final plateau phase. This behavior alters the slope value, which is calculated by averaging the phases, which will certainly have very different slopes.

In contrast, ColorTV exhibits a consistent and continuous gap reduction, remaining below the theoretical rate $\mathcal{O}(1/\sqrt{k})$ for most of the optimization run and reaching gap values well below 10^{-2} at later epochs.

So, to get a more complete view of the convergence behavior, we complement the slope fit with threshold-based iteration counts. The normalized gap is defined as:

$$\psi_k = \frac{L_k^{\text{best}} - \bar{f}_\lambda}{L_0^{\text{best}} - \bar{f}_\lambda},$$

and Table 4.4 reports the first epoch at which ψ_k falls below 0.10, 0.05, and 0.01 (corresponding to 90%, 95%, and 99% reduction of the initial empirical gap).

The threshold analysis highlights a clear advantage for ColorTV in practical convergence speed. For $\lambda \leq 10^{-4}$, ColorTV reaches the 10% threshold in ≈ 1000 iterations, compared to ≈ 2000 iterations for SGPTL. Similarly, at the 5% threshold, ColorTV requires between 2907 and 3302 iterations, whereas SGPTL requires between 4038 and 4742 iterations. Furthermore, ColorTV successfully achieves the stringent 1% threshold ($\psi \leq 0.01$) for most tested λ values, whereas SGPTL fails to reach this accuracy within the maximum epoch budget for all $\lambda \leq 10^{-3}$ due to its premature stagnation.

At $\lambda = 10^{-2}$, both methods reach all three thresholds, yet ColorTV remains substantially faster, hitting the 1% target in 1300 iterations compared to 7945 iterations for SGPTL.

λ	ColorTV		SGPTL	
	Slope (s)	R^2	Slope (s)	R^2
10^{-7}	-0.713	0.987	-0.845	0.880
10^{-6}	-0.761	0.976	-0.864	0.883
10^{-5}	-0.715	0.992	-0.875	0.908
10^{-4}	-0.792	0.965	-0.838	0.898
10^{-3}	-1.110	0.966	-1.032	0.881
10^{-2}	-1.889	0.969	-1.156	0.588

Table 4.3: Empirical decay slopes (s) and goodness-of-fit (R^2) of the best-known gap.

λ	$\psi \leq 0.10$		$\psi \leq 0.05$		$\psi \leq 0.01$	
	ColorTV	SGPTL	ColorTV	SGPTL	ColorTV	SGPTL
10^{-7}	1054	2027	3302	4038	–	–
10^{-6}	976	2077	2955	4182	17696	–
10^{-5}	953	2097	2907	4287	–	–
10^{-4}	920	2127	3153	4742	14943	–
10^{-3}	1193	2395	1961	3234	6981	–
10^{-2}	410	6041	546	6605	1300	7945

Table 4.4: Number of iterations required to reduce the empirical best-known gap to 10%, 5%, and 1% of its initial value.

“–” indicates that the threshold was not reached within the training budget.

As a final observation, the results indicate that the ColorTV stepsize rule allows to obtain better convergence values, with a smoother convergence curve, significantly faster in the final phase of convergence.

4.1.3 Verification of the Expected Theoretical Properties of Subgradient Methods

Some plot in the following section were performed fixing the regularization parameter of λ . The same qualitative behaviors can be observed across all tested values of λ , although their quantitative magnitude varies with the regularization strength. Furthermore, these results are analyzed using the ColorTV stepsize rule.

λ	Volume Algorithm	Heavy Ball
10^{-7}	0.0190	0.4272
10^{-6}	0.0287	0.4110
10^{-5}	0.0295	0.4155
10^{-4}	0.0376	0.4213
10^{-3}	0.2090	0.4337
10^{-2}	0.2802	0.4839

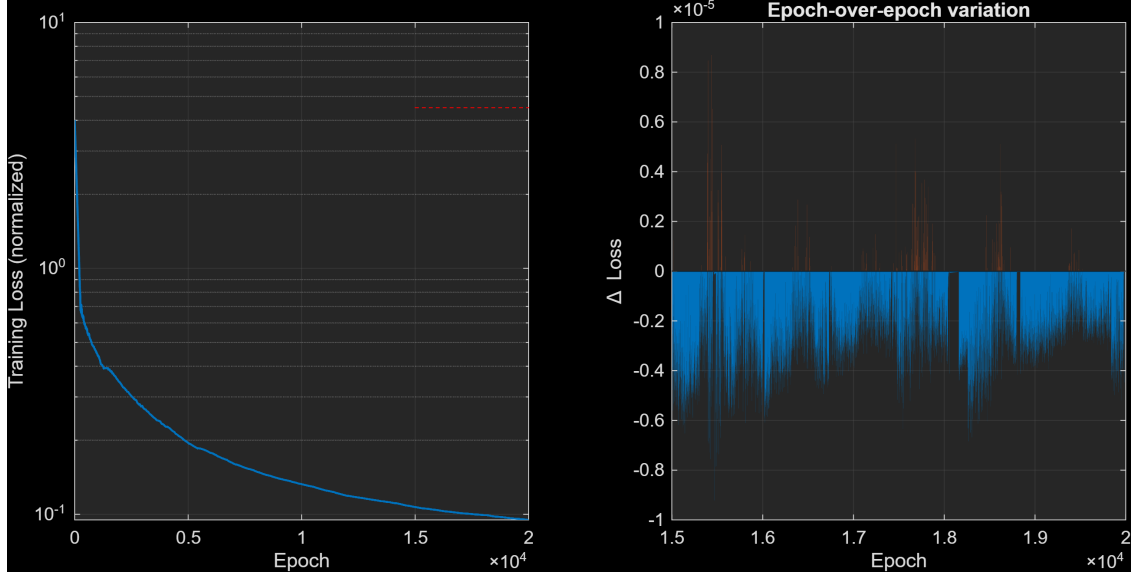
Table 4.5: Comparison of mean loss oscillation between the Volume Algorithm and Heavy Ball.

Non-monotonicity of the subgradient As observed in the training loss curves (Fig. 3.1a and Fig. 3.4a) and the quantitative oscillation analysis across β values (Fig. 3.1b and Fig. 3.4b), optimization via subgradient methods exhibits non-monotonic behavior. Particularly in the final stages of training, the loss trajectory displays transient intervals of local increase. This aligns with theoretical expectations: unlike gradients in smooth optimization, a subgradient is not guaranteed to be a strict local descent direction (Sec. 2.3).

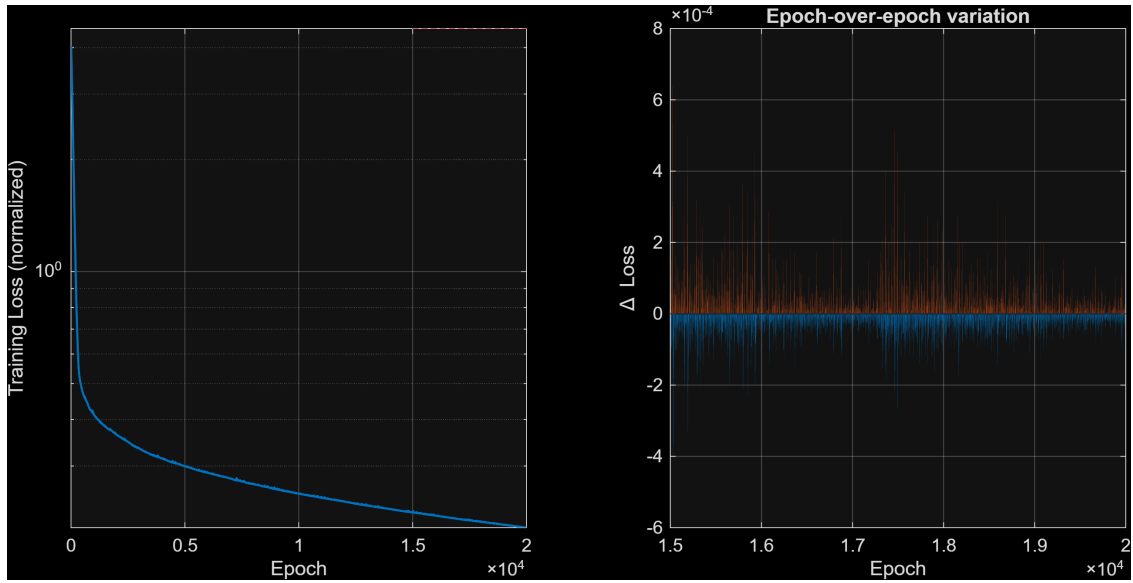
The results in Table 4.5 and the qualitative comparison in Figure 4.2 (for $\lambda = 10^{-4}$) demonstrate that the Volume Algorithm drastically reduces these oscillations compared to the Heavy Ball momentum method. In general, under low regularization values ($\lambda \leq 10^{-4}$), the mean trajectory oscillation of deflection algorithm is extremely reduced compared to the Heavy-Ball algorithm. This advantage is lower but remains substantial for values of $\lambda \geq 10^{-3}$, where the oscillation of the Volume algorithm is roughly halved compared to Heavy Ball mean oscillation.

The observed reduction is consistent with the role of the deflection mechanism, which convexly combines information from previous subgradient directions rather than directly accumulating updates through a velocity momentum term. In the tested configurations, this results in a substantially smoother and more stable optimization trajectory which also leads to better optimization results.

Comparison with Subgradient without Deflection To investigate the contribution of the subgradient deflection mechanism in the Volume Algorithm, an experiment was conducted by running the same model – using identical initialization weights and hyperparameter values –



(a) Training loss trajectory using the Volume Algorithm with deflection ($\lambda = 10^{-4}$).



(b) Training loss trajectory using the Heavy Ball method ($\lambda = 10^{-4}$).

Figure 4.2: Comparison of local loss oscillations between the Volume Algorithm and Heavy Ball for $\lambda = 10^{-4}$, illustrating the dampening effect of subgradient deflection.

both with and without the deflection rule. The results, summarized in Table 4.6, show a substantial reduction in trajectory oscillation when deflection is enabled for most of the tested regularization values, in addition to the much lower loss values achieved by the algorithm with deflection.

- **Damping of oscillations:** Without deflection, the optimization process exhibits persistent, large-scale oscillations across all tested values of λ . The mean oscillation increases from the range 0.0190 - 0.2802 achieved with deflection to 0.2604 - 0.3757 achieved without deflection, supporting the stabilizing effect of the deflection mechanism in the tested configurations, as reported in the literature.
- **Superiority in terms of cost function:** Deflection leads to lower final objective values for all values of λ . The difference is particularly pronounced for small regularization values, where the deflection-enabled method reaches substantially lower losses, whereas, for higher values of λ , the regularization term dominates the loss and reduces the gap between the two implementations.

λ	With deflection		Without deflection	
	Loss	Osc.	Loss	Osc.
10^{-7}	0.0478	0.0190	0.1934	0.2643
10^{-6}	0.0363	0.0287	0.1944	0.2662
10^{-5}	0.0536	0.0295	0.2083	0.2637
10^{-4}	0.0949	0.0376	0.2610	0.2604
10^{-3}	0.3015	0.2090	0.4167	0.2788
10^{-2}	0.6810	0.2802	0.7020	0.3757

Table 4.6: Comparison between subgradient method with and without the deflection mechanism.

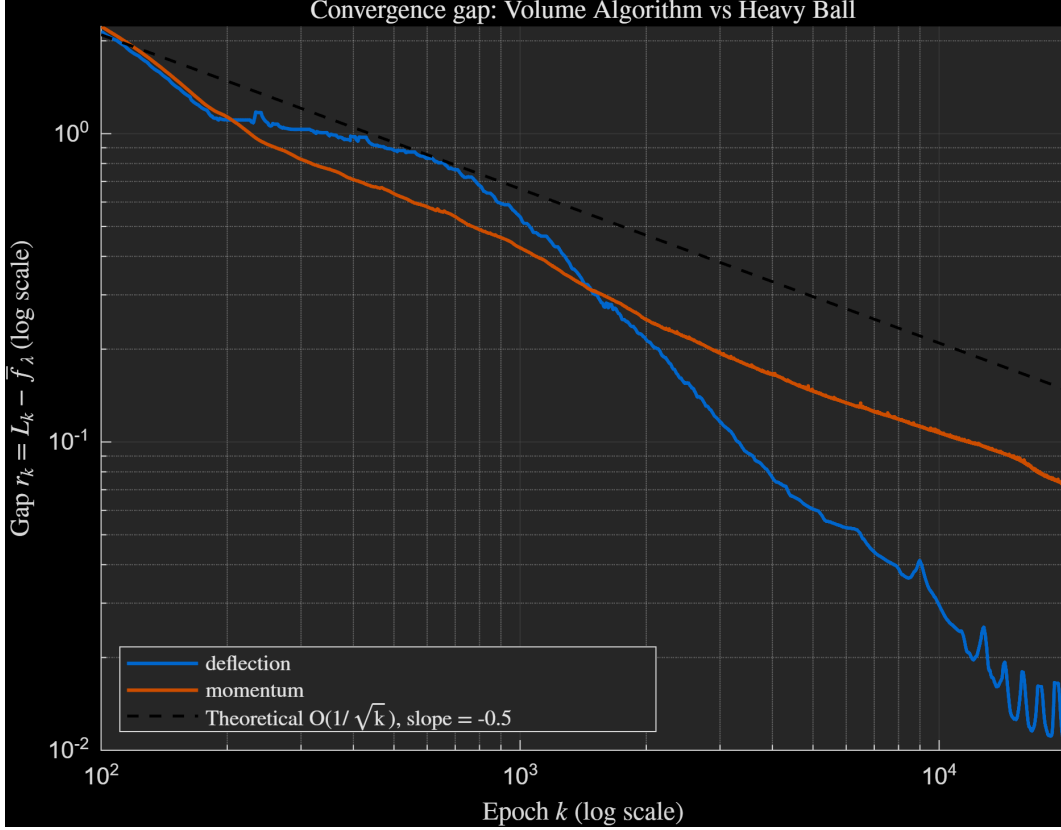


Figure 4.3: Convergence gap comparison between the Volume Algorithm (deflection) and Heavy Ball (momentum) for $\lambda = 10^{-3}$

Convergence speed As shown in Table 4.7, log-log linear regression—implemented as described in Sec. 4.1 reveals that the Volume Algorithm achieves steeper convergence slopes compared to Heavy Ball across almost all regularization regimes. Specifically, the Volume Algorithm produces a considerably steeper decay slope for higher regularization values ($\lambda \geq 10^{-3}$), whereas its slope magnitude remains around $|s| \approx 0.7$ for smaller regularization values ($\lambda \leq 10^{-4}$). In contrast, Heavy Ball exhibits similar and lower slopes values across all regimes; even for $\lambda = 10^{-2}$, where its decay accelerates significantly ($s = -1.269$), it remains lower in magnitude than the corresponding slope of the Volume Algorithm ($s = -1.889$).

We can notice how the Volume Algorithm’s slopes consistently exceed the theoretical $\mathcal{O}(1/\sqrt{k})$ benchmark ($|s| = 0.5$) (valid for convex problems), whereas Heavy Ball’s slopes (excluding $\lambda = 10^{-2}$) remain closer to, and in most cases below, this theoretical rate.

This crossover is directly visible in Fig. 4.3: while both algorithms initially follow a common trajectory, Heavy Ball’s gap decay visibly flattens after $k \approx 10^3$, whereas the Volume Algorithm continues its steeper descent.

As previously discussed in Sec. 4.1.2, the estimated global slope does not fully characterize the local dynamics of the convergence trajectory. To complement this analysis, threshold-based measurements relative to the best known empirical gap were evaluated.

The results reported in Table 4.8 reveal a more nuanced behavior. For nearly all values of $\lambda \leq$

10^{-3} , Heavy Ball reaches the coarse threshold ($\psi \leq 0.10$) in significantly fewer iterations than the Volume Algorithm. However, this initial advantage is drastically reversed as higher precision is demanded: Heavy Ball fails to reach the strict threshold ($\psi \leq 0.01$) within the maximum epoch budget for all $\lambda \leq 10^{-3}$. Even for $\lambda = 10^{-2}$, Heavy Ball requires 1840 iterations to achieve $\psi \leq 0.01$, compared to just 1300 for the Volume Algorithm.

Heavy Ball exhibits rapid initial progress followed by persistent stagnation near non-smooth regions, while, indeed, the Volume Algorithm maintains a more sustained decrease over the analyzed interval, despite an initial lower decrease compared to Heavy Ball.

Theoretical interpretation These empirical results align well with optimization theory for non-smooth functions:

1. **Limitations of Heavy Ball in non-smooth optimization:** The Heavy Ball momentum is originally formulated for smooth convex functions with Lipschitz-continuous gradients. In non-smooth contexts, subgradients undergo abrupt, discontinuous changes in direction. Standard Heavy Ball momentum accumulates these orthogonal components, causing transverse oscillations across the optimality valley that hinder the asymptotic decay rate.
2. **Effectiveness of subgradient deflection:** In contrast, the Volume Algorithm constructs a deflected search direction d_k via a convex combination of previous subgradients while adhering to a strict step-size schedule. This mechanism filters out high-frequency transverse oscillations, generating a stable direction to the optimal set X^* .

λ	Volume Algorithm		Heavy Ball	
	Slope (s)	R^2	Slope (s)	R^2
10^{-7}	-0.713	0.987	-0.391	0.971
10^{-6}	-0.761	0.976	-0.400	0.972
10^{-5}	-0.715	0.992	-0.400	0.970
10^{-4}	-0.792	0.965	-0.354	0.962
10^{-3}	-1.110	0.966	-0.562	0.989
10^{-2}	-1.889	0.969	-1.269	0.957

Table 4.7: Empirical convergence slopes (s) and goodness-of-fit (R^2) comparison between Volume Algorithm and Heavy Ball.

λ	$\psi \leq 0.10$		$\psi \leq 0.05$		$\psi \leq 0.01$	
	Deflection	Momentum	Deflection	Momentum	Deflection	Momentum
10^{-7}	1054	574	3302	6006	–	–
10^{-6}	976	547	2955	5867	17696	–
10^{-5}	953	538	2907	5918	–	–
10^{-4}	920	591	3153	7547	14943	–
10^{-3}	1193	950	1961	2407	6981	–
10^{-2}	410	613	546	1077	1300	1840

Table 4.8: Number of iterations required to reduce the empirical gap to 10%, 5%, and 1% of its initial value.

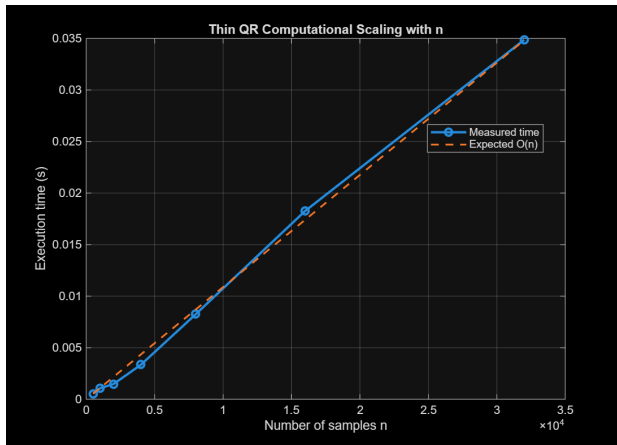
“–” indicates that the threshold was not reached within the training budget.

4.2 Model M2: Considerations on QR direct method

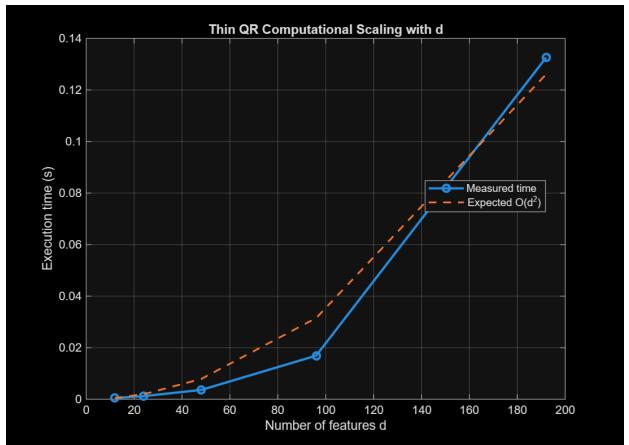
4.2.1 Computational performance

For the present problem, the training matrix contains a relatively small number of features and samples. The thin QR factorization performed through Householder reflectors has computational complexity $\mathcal{O}(nd^2)$ for $n \gg d$. Given the small value of d , the computational cost of the factorization is therefore negligible.

To verify the actual computational complexity of the direct method, several runs of algorithm A3 were performed while varying separately n and then d . After that, the execution times have been plotted along with the asymptotic theoretical limits for comparison (Fig.4.4a and 4.4b):



(a) Scaling n vs $\mathcal{O}(n)$



(b) Scaling d vs $\mathcal{O}(d^2)$

Figure 4.4: Plots of execution times of algorithm A3 with scaling of n and d .

n	Execution time (s)
500	0.000544
1000	0.001067
2000	0.001482
4000	0.003353
8000	0.008248
16000	0.018269
32000	0.034868

Table 4.9: Execution times of A3 with scaling n and fixed $d = 12$.

d	Execution time (s)
12	0.000492
24	0.001232
48	0.003622
96	0.016923
192	0.132584

Table 4.10: Execution times of A3 with scaling d and fixed $n = 500$.

As demonstrated in Figure 4.4, the empirical execution times follow the theoretical complexity limits. Figure 4.4a confirms that when the number of features is fixed, the execution time grows linearly with the number of samples, mirroring almost perfectly the expected $\mathcal{O}(n)$ asymptotic behavior. Conversely, Figure 4.4b illustrates that scaling the features while keeping the sample size fixed yields a quadratic curve, aligning not perfectly but closely enough with the theoretical $\mathcal{O}(d^2)$ complexity.

4.2.2 Numerical stability and Conditioning verification

As introduced in Section 3.1.2, the formation of normal equations squares the conditioning number, $\kappa(X^\top X) = \kappa(X)^2 \approx 1036$, and the regularization reduces it to $\kappa(X^\top X + n\lambda^* I) \approx 104.0$ for $\lambda^* = 0.1$ (Table 3.2). Algorithm A3 avoids this squaring by directly factoring the augmented matrix $\tilde{X}$. Since

$$\tilde{X}^\top \tilde{X} = X^\top X + n\lambda I, \quad (4.1)$$

the singular values of $\tilde{X}$ are the square roots of the eigenvalues of the regularized matrix, and therefore

$$\kappa(\tilde{X}) = \sqrt{\kappa(X^\top X + n\lambda I)} \approx \sqrt{104.0} \approx 10.20 \quad (\lambda = \lambda^*). \quad (4.2)$$

For $\lambda \rightarrow 0$, this quantity tends to $\kappa(X) \approx 32.2$, so regularization reduces the conditioning of the problem solved by A3 from about 32.2 to about 10.2, while the normal equations would go from about 1036 to about 104. Figure 4.5 shows both quantities as functions of λ : the curve of $\tilde{X}$ is always the square root of the curve of the normal equations, and the difference between them is greatest for small values of λ , which is the regime in which collinearity is most relevant.

Householder QR is a backward-stable factorization method, making it appropriate for the regularized least squares problem in the presence of collinearity. This is empirically confirmed by the orthogonality error of the calculated value of Q_1 :

$$\|Q_1^\top Q_1 - I\|_F = 7.406256 \cdot 10^{-15}, \quad (4.3)$$

which is at the level of machine precision.

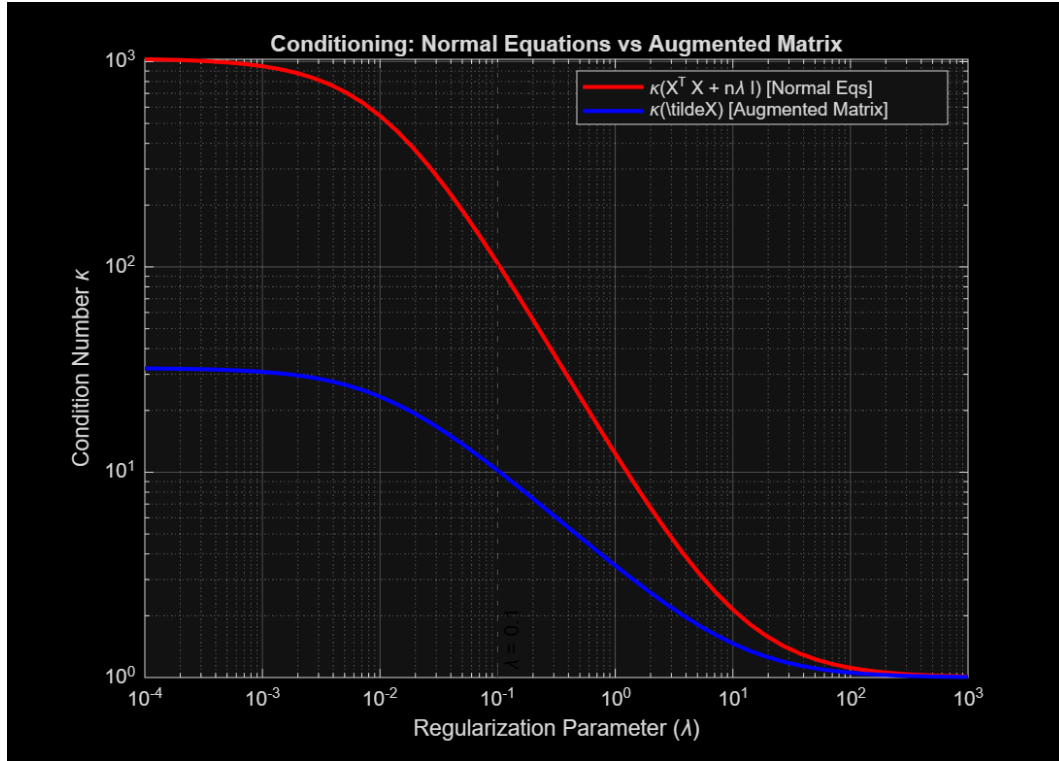


Figure 4.5: Condition number of the Normal Equations versus the Augmented Matrix across varying λ .

4.2.3 Effect of L2 Regularization

The choice of $\lambda^* = 0.1$ influences the structure of the resulting parameter matrix $\Theta \in \mathbb{R}^{13 \times 4}$. The L2 penalty reduces the magnitude of the coefficients and decreases the sensitivity of the solution to collinearity in the input features, as quantified by the conditioning improvement in Section 3.1.2.

Figure 4.6 shows the effect of the L2 penalty on the elements of Θ . Given the strong collinearity of the dataset, the unregularized solution can assign large weights of opposite signs to nearly redundant features. As λ increases towards $\lambda^* = 0.1$, the magnitude of the coefficients gradually decreases, and all converge to zero for large values of λ . This trace shows that $\lambda = 0.1$ applies sufficient regularization to anchor the parameters and mitigate collinear variance without excessively flattening the model's expressive capacity.

λ	MSE train
0	0.64400
10^{-4}	0.64400
10^{-3}	0.64407
10^{-2}	0.64658
10^{-1}	0.65817
1	0.66768
10	0.73946
10^2	0.93365
10^3	0.99057
10^4	0.99725

Table 4.11: Training MSE of Thin QR as λ varies.

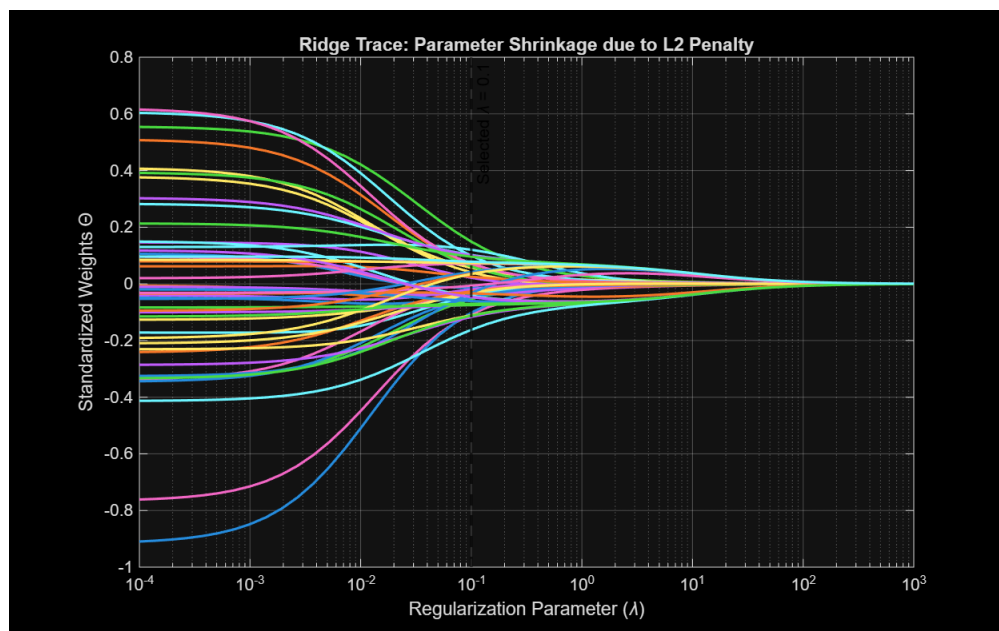


Figure 4.6: Ridge Trace plot demonstrating the shrinkage of standardized feature weights as λ increases.

Bibliography

- [1] A. Frangioni, B. Gendron, E. Gorgone, *On the computational efficiency of subgradient methods: a case study with Lagrangian bounds*, 2017.
Available: <https://doi.org/10.1007/s12532-017-0120-7>
- [2] F. Barahona, R. Anbil, The volume algorithm: producing primal solutions with a subgradient method, *Mathematical Programming*, 87(3):385–399, 2000.
Available: <https://doi.org/10.1007/s101070050002>
- [3] B. T. Polyak, *Minimization of unsmooth functionals*, 1969.
Available: https://www.researchgate.net/publication/238270077_Minimization_of_Unsmooth_Functionals
- [4] P. Ochs, Local Convergence of the Heavy-Ball Method and iPiano for Non-convex Optimization, *Journal of Optimization Theory and Applications*, 177, 153–180 (2018).
- [5] G. d’Antonio, A. Frangioni, Convergence Analysis of Deflected Conditional Approximate Subgradient Methods, *SIAM Journal on Optimization* 20(1), 357–386, 2009.
Available: <http://pages.di.unipi.it/frangio/abstracts.html#SIOPT08>
- [6] F. H. Clarke, *Optimization and Nonsmooth Analysis*, 1987.
Available: <https://epubs.siam.org/doi/book/10.1137/1.9781611971309>
- [7] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010
- [8] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification, *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [9] B. T. Polyak, Some methods of speeding up the convergence of iteration methods, *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
Available: <https://papers.baulab.info/papers/also/Polyak-1964.pdf>

- [10] S. Bubeck, Convex Optimization: Algorithms and Complexity, *Foundations and Trends in Machine Learning*, vol. 8, no. 3-4, pp. 231–358, 2015.
Available: <https://arxiv.org/abs/1405.4980>
- [11] J. W. Demmel, *Applied Numerical Linear Algebra*, SIAM, 1997, Ch.3,
Available: <https://www.stat.uchicago.edu/~lekheng/courses/309/books/Demmel.pdf>